

Physics Embedded Transformers: Module-Level Documentation

Project Documentation

July 22, 2026

Contents

1	Repository Architecture	6
1.1	Top-Level Layout	6
1.2	Physical Device	6
2	Configuration Module	7
2.1	Files	7
2.2	Scientific Role	7
2.3	Main Interface	7
2.4	Validation Behavior	8
2.5	Physical Parameters in Current Configs	8
3	Physics Package Overview	8
3.1	Package Layout	8
3.2	Data Flow	9
4	Physics Geometry: Coordinate and Vector Conventions	9
4.1	Project-Wide Rule	9
4.2	Coordinate Frames	9
4.3	Point Transforms	10
4.4	Vector Transforms	10
4.5	Observed Droplet Velocities	10
4.6	CFD Direction Audit	10
5	Physics Geometry: Centerline Module	11
5.1	Files	11
5.2	Physical Significance	12
5.3	Coordinate and Unit Convention	12
5.4	Implementation Details	12
5.5	Outputs	13
5.6	Execution	14
6	Physics Geometry: Device Region Labeling	14
6.1	Files	14
6.2	Physical Significance	15
6.3	Governing Geometry	15
6.4	Implementation Details	15

6.5	Outputs	16
6.6	Execution	16
7	Physics Occupancy: Droplet Fractional Occupancy	17
7.1	Files	17
7.2	Physical Significance	17
7.3	Droplet Footprint Model	17
7.4	Raw and Normalized Occupancy	17
7.5	Implementation Details	18
7.6	Outputs	18
7.7	Execution	21
8	Physics Hydraulics: Baseline Branch Model	22
8.1	Files	22
8.2	Physical Significance	22
8.3	Effective Droplet Resistance	22
8.4	Flow Split	22
8.5	Superficial Mixture Velocity	23
8.6	Implementation Details	23
8.7	Outputs and Diagnostics	23
8.8	Execution	24
9	Physics CFD: Full-Device Stokes Model	24
9.1	Scientific Role	24
9.2	Implemented Files	25
9.3	Configuration	25
9.4	Geometry Construction	25
9.5	Mesh Generation	27
9.6	Governing Equations	30
9.7	Boundary Conditions	30
9.8	Finite-Element Discretization	31
9.9	Validated $\alpha = 0$ Solution	31
9.10	Scalar Mesh Convergence	34
9.11	Field-Level Mesh Verification	35
9.12	Flux Diagnostic	37
9.13	Production Decision	37
9.14	Execution	37
10	Physics Interpolation: Full-Device Velocity Fields	38
10.1	Architectural Role	38
10.2	Files	38
10.3	Interpolation Model	39
10.4	Sampling	39
10.5	Validation	40
10.6	Execution	41

11 Physics Enrichment: Tracking Table With Hydraulic and CFD Context	42
11.1 Purpose	42
11.2 Immutable Inputs	42
11.3 Files	42
11.4 Hydraulic-State Join	43
11.5 Coordinate Mapping	43
11.6 CFD Background Sampling	43
11.7 Output Schema	43
11.8 Missing-Field Semantics	44
11.9 Validation Results	44
11.10 Execution	46
12 Physics State Placeholder	47
12.1 Files	47
12.2 Future Scientific Role	47
13 Physics-Embedded Markovian Rollout Model	47
13.1 Scientific Purpose	47
13.2 State Definition	47
13.3 Autoregressive Rollout	48
13.4 Loss Masking	48
13.5 Implementation	48
13.6 Attention and Trajectory Diagnostics	49
14 Validation: Droplet Speed versus Branch Velocity	49
14.1 Files	49
14.2 Scientific Question	49
14.3 Measured Speed from Centered Difference	50
14.4 Branch-Interior Filtering	50
14.5 Normalization and Correlation	50
14.6 Candidate Selection	50
14.7 Current Results	51
14.8 Outputs and Figures	51
14.9 Execution	57
15 Rollout Model Comparison	58
15.1 Purpose	58
15.2 Models Compared	58
15.3 Rollout Semantics	58
15.4 Metrics	59
15.5 Final Results	59
15.6 Figures	60
15.7 Execution	62
16 Testing and Execution	62
16.1 Test Suite	62
16.2 Primary Commands	63
16.3 Current Scope Boundaries	64

List of Figures

1	Full-device CFD velocity vectors in device Cartesian coordinates.	11
2	Calibrated centerline geometry diagnostic. The plotted branches encode the physical inlet, left branch, right branch, and outlet topology.	14
3	Device-level physical region overlay. This image shows the precomputed region labels used by droplet occupancy. Regions are not recomputed by the occupancy or validation visualizers.	16
4	Historical centerline-region diagnostic for frame 0. It is retained as a development figure; the production occupancy path uses the device-level labels in Figure 3.	16
5	Occupancy diagnostics summarizing physical coverage, active regions, boundary clipping, and normalized occupancy behavior.	19
6	Representative droplet occupancy spot checks. These verify ellipse rasterization against the precomputed device region labels.	20
7	Manual inspection montage for low-coverage or previously invalid occupancy cases.	21
8	Baseline branch hydraulics diagnostics. The plot shows effective branch occupancy, effective length, superficial velocity, and flow/occupancy relationships.	24
9	Full-device fluid polygon reconstructed from calibrated centerlines. The blue region is the continuous fluid domain and the white central island is the no-slip solid boundary.	26
10	Zoomed CAD diagnostics at the inlet split and outlet merge. These checks verify that the branch transitions are continuous and do not contain internal wall remnants.	27
11	Frozen 24 μm full-device triangular mesh.	28
12	Mesh aspect-ratio diagnostic for the full-device production mesh.	29
13	Production mesh near the inlet split and outlet merge.	30
14	Velocity magnitude for the validated full-device $\alpha = 0$ Stokes solution.	32
15	Pressure field for the validated full-device $\alpha = 0$ Stokes solution.	33
16	Dense post-processed streamlines sampled from the solved full-device velocity field. These streamlines are visualization samples of a continuous FEM velocity field, not a limit on CFD resolution.	34
17	Scalar mesh convergence for the full-device $\alpha = 0$ Stokes solution.	35
18	Velocity magnitude on the common grid for the 24 μm candidate and 12 μm reference. Both panels use identical color limits.	36
19	Absolute vector error and angular error between the 24 μm and 12 μm full-device velocity fields.	36
20	Field-level error zooms at the full-device inlet split and outlet merge. Both fields come from the complete loop geometry.	37
21	Withheld-case interpolation errors versus left-flow fraction. The errors are computed from FEM velocity coefficients and direct physical-domain samples, not from plotted images.	41
22	Representative withheld-case validation for $\alpha = 0.40$: interpolated speed, speed error, and angular error away from low-speed regions.	41
23	Representative tracked-droplet overlay on the CFD junction mesh. Inside-domain droplets are distinguished from outside-domain droplets, and sampled background velocity vectors are shown for covered droplets.	45
24	Number of tracked droplets inside the CFD domain as a function of frame.	46
25	Angular difference between observed droplet motion direction and sampled background-flow direction for droplets inside the CFD patch.	46

26	Pooled normalized comparison of calculated branch superficial velocity and measured droplet speed. The pooled correlation is descriptive because repeated frames within a trajectory are temporally correlated.	52
27	Selected left-branch validation track 3463.	53
28	Selected left-branch validation track 254.	53
29	Selected left-branch validation track 3127.	54
30	Selected left-branch validation track 1713.	54
31	Selected left-branch validation track 3460.	55
32	Selected right-branch validation track 3461.	55
33	Selected right-branch validation track 257.	56
34	Selected right-branch validation track 3465.	56
35	Selected right-branch validation track 3462.	57
36	Selected right-branch validation track 1712.	57
37	Position RMSE over a 50-step rollout for the two previous Markovian baselines and the physics-embedded Markovian model.	60
38	Velocity RMSE over the same aligned validation windows.	61
39	Fraction of predicted droplet footprints with more than 2% of the ellipse outside the channel mask.	61
40	Junction branch-decision outcomes for true junction events.	62

1 Repository Architecture

The repository is organized around a scientific preprocessing and physics stack for a microfluidic asymmetric-loop device. The architecture now follows the physical workflow:

Geometry $\rightarrow$ Occupancy $\rightarrow$ Hydraulics $\rightarrow$ CFD $\rightarrow$ Physics State.

The code implementing this workflow lives under:

```
src/physics/  
  geometry/  
  occupancy/  
  hydraulics/  
  cfd/  
  state/
```

The current implementation is deterministic and physics-preparatory. It does not optimize temporal lag, fit hydraulic parameters, or resolve droplets as moving multiphase CFD objects. It does include a validated steady Stokes full-device CFD solve for the continuous phase, a physics-enriched canonical training dataset, a physics-embedded Markovian rollout model, and a self-contained rollout comparison against the two previous Markovian baselines.

1.1 Top-Level Layout

Path	Scientific responsibility
src/config/	Resolve experiment and device configuration.
src/physics/geometry/	Device centerlines, physical geometry, and region labels.
src/physics/occupancy/	Droplet fractional occupancy over physical regions.
src/physics/hydraulics/	Baseline branch-resistance and superficial-velocity model.
src/physics/full_device_cfd/	Full-loop CAD geometry, mesh generation, Stokes solve, and field verification.
src/physics/state/	Placeholder for future unified physics state.
src/evaluation/	Self-contained rollout comparison bundle for baseline and physics-embedded models.
src/datasets/	Canonical dataset builders and window loaders.
src/models/	Transformer model architecture used by the physics Markovian trainer.
scripts/geometry/	Geometry and device-region command-line tools.
scripts/occupancy/	Droplet occupancy command-line tools.
scripts/physics/	Hydraulics and CFD command-line tools.
scripts/training/	Physics Markovian training and attention visualization entry points.
scripts/validation/	Standalone scientific validation analyses.

1.2 Physical Device

The present device is an asymmetric microfluidic loop with channel width and height both set to 100 μm . The image calibration is:

$$1 \text{ px} = 4 \mu\text{m}.$$

The loop branch lengths are:

$$L_{\text{left}} = 1791 \mu\text{m}, \quad L_{\text{right}} = 1491 \mu\text{m}.$$

Thus the left branch is the long branch and the right branch is the short branch:

$$\frac{L_{\text{left}}}{L_{\text{right}}} = 1.201207, \quad \alpha = \frac{L_{\text{left}} - L_{\text{right}}}{L_{\text{right}}} = 0.201207.$$

2 Configuration Module

2.1 Files

- `src/config/loader.py`
- `src/config/__init__.py`
- `configs/devices/asymmetric_loop_h100.yml`
- `configs/experiments/video.2.yml`
- `configs/physics/junction_cfd.yml`

2.2 Scientific Role

The configuration layer separates physical device properties from experiment conditions. This is important because device geometry, calibration, and channel dimensions are fixed properties of the microfluidic chip, whereas flow rates, tracked data, and validation outputs are experiment-specific.

2.3 Main Interface

```
load_experiment_config(
    experiment_path: str | Path,
    configs_root: str | Path = "configs",
) -> dict
```

The loader reads an experiment YAML file, extracts `experiment.device_id`, resolves the linked device file as:

```
configs/devices/<device_id>.yml
```

It verifies:

```
device.id = experiment.device_id.
```

The returned structure is:

```
{
  "experiment": <full experiment YAML mapping>,
  "device": <full device YAML mapping>
}
```

2.4 Validation Behavior

The loader raises clear exceptions for missing experiment files, malformed or empty YAML, missing `experiment.device_id`, missing linked device files, and mismatched device IDs.

2.5 Physical Parameters in Current Configs

Quantity	Value
Device ID	<code>asymmetric_loop_h100</code>
Experiment ID	<code>exp_0002</code>
Pixel scale	4.0 $\mu\text{m}/\text{px}$
Channel width	100 μm
Channel height	100 μm
Continuous flow rate	1950 $\mu\text{L}/\text{hr}$
Dispersed flow rate	10 $\mu\text{L}/\text{hr}$
Total mixture flow rate	1960 $\mu\text{L}/\text{hr}$

3 Physics Package Overview

The physics package groups all scientific transformations that convert raw experimental tracking and device geometry into physically meaningful state variables. Its present responsibilities are:

1. reconstruct a calibrated device geometry from centerlines;
2. label physical channel regions at the device level;
3. compute droplet occupancy fractions in those regions;
4. compute baseline branch hydraulic state from occupancy;
5. generate a CFD-ready full-device domain, finite-element mesh, and validated steady Stokes velocity field;
6. interpolate the full-device CFD library onto tracked droplet centroids;
7. produce a physics-enriched canonical dataset for Markovian rollout training;
8. reserve an interface for future unified physics state.

3.1 Package Layout

```
src/physics/  
  geometry/  
    centerlines.py  
    regions.py  
    types.py  
  occupancy/  
    ellipse.py  
    calculator.py
```



```

hydraulics/
  baseline.py
full_device_cfd/
  domain.py
  mesh.py
  solver.py
  field_verification.py
state/
  physics_state.py

```

3.2 Data Flow

```

centerline CSV → calibrated centerline geometry
                → device region label map
                → droplet occupancy vectors
                → baseline branch hydraulics
                → full-device CFD velocity fields.

```

The transformer is intentionally outside this package. It should consume outputs from the physics package, not own the physical preprocessing itself. The first physics consumer is the 15-feature Markovian rollout model documented in the model section.

4 Physics Geometry: Coordinate and Vector Conventions

4.1 Project-Wide Rule

No array or column may be named only x , y , vx , or vy at a boundary between physics modules unless its coordinate frame and units are defined by the enclosing typed object. Points and vectors must be transformed separately.

4.2 Coordinate Frames

Image frame. The image frame uses x_{px} and y_{px} . The origin is the top-left video pixel, $+x$ points right, $+y$ points downward, and units are pixels.

Device Cartesian frame. The device frame uses x_{device_um} and y_{device_um} . The origin is fixed by the lower-left image reference line, $+x$ points right, $+y$ points upward, and units are micrometers. For `video_2`, the fixed reference is taken from the device region metadata image height:

$$y_{reference,px} = 596.$$

Full-device CFD frame. The full-device CFD mesh uses the same physical orientation as the device Cartesian frame. Thus x_{cfd_um} and y_{cfd_um} are micrometer coordinates with $+x$ to the right and $+y$ upward. This keeps CAD construction, FEM assembly, field comparison, and downstream sampling in a single physical coordinate convention.

4.3 Point Transforms

For pixel scale s in micrometers per pixel,

$$x_{\text{device}} = sx_{\text{px}}, \quad y_{\text{device}} = s(y_{\text{reference,px}} - y_{\text{px}}).$$

The full-device CFD mesh uses the device Cartesian coordinate system, so

$$x_{\text{cfd}} = x_{\text{device}}, \quad y_{\text{cfd}} = y_{\text{device}}.$$

4.4 Vector Transforms

Vectors use only the linear part of the affine transform. Translation is never applied to a vector:

$$v_{x,\text{device}} = sv_{x,\text{image}}, \quad v_{y,\text{device}} = -sv_{y,\text{image}}.$$

Between device Cartesian and full-device CFD frames:

$$u_{x,\text{cfd}} = u_{x,\text{device}}, \quad u_{y,\text{cfd}} = u_{y,\text{device}}.$$

The inverse vector transform is identical:

$$u_{x,\text{device}} = u_{x,\text{cfd}}, \quad u_{y,\text{device}} = u_{y,\text{cfd}}.$$

4.5 Observed Droplet Velocities

The current tracked-feature file contains framewise centroid positions and no canonical physical velocity columns. Centered finite differences of `centroid_x` and `centroid_y` produce image-frame velocities in pixels per frame. The device-frame conversion is:

$$v_{x,\text{device}} = sv_{x,\text{image}}, \quad v_{y,\text{device}} = -sv_{y,\text{image}}.$$

No frame-rate conversion is currently applied in the enrichment direction audit, so observed droplet speeds remain in micrometers per frame. Direction comparisons are still valid because both observed and CFD vectors are converted to the same device Cartesian frame before computing angles.

4.6 CFD Direction Audit

The coordinate audit evaluates the full-device CFD field directly at representative inlet, branch, and outlet points. For a point $\mathbf{x}$, a local downstream reference point $\mathbf{x}_d$, and velocity $\mathbf{u}$, the geometric test is:

$$\mathbf{u} \cdot (\mathbf{x}_d - \mathbf{x}) > 0.$$

Region	x_{cfd}	y_{cfd}	u_x	u_y	Expected direction
inlet	device frame	device frame	u_x	u_y	toward upper split
left branch	device frame	device frame	u_x	u_y	around long arm
right branch	device frame	device frame	u_x	u_y	around short arm
outlet	device frame	device frame	u_x	u_y	downstream from lower merge

The signed boundary fluxes are:

$$Q_{\text{inlet}} = -5.444444 \times 10^{-6} \text{ m}^2/\text{s}, \quad Q_{\text{outlet}} = 5.477429 \times 10^{-6} \text{ m}^2/\text{s}.$$

The negative inlet sign follows the outward-normal convention: flow enters the domain at the inlet and leaves through the outlet. The full-device CFD field is therefore physically oriented in the same Cartesian frame used by the geometry and mesh.

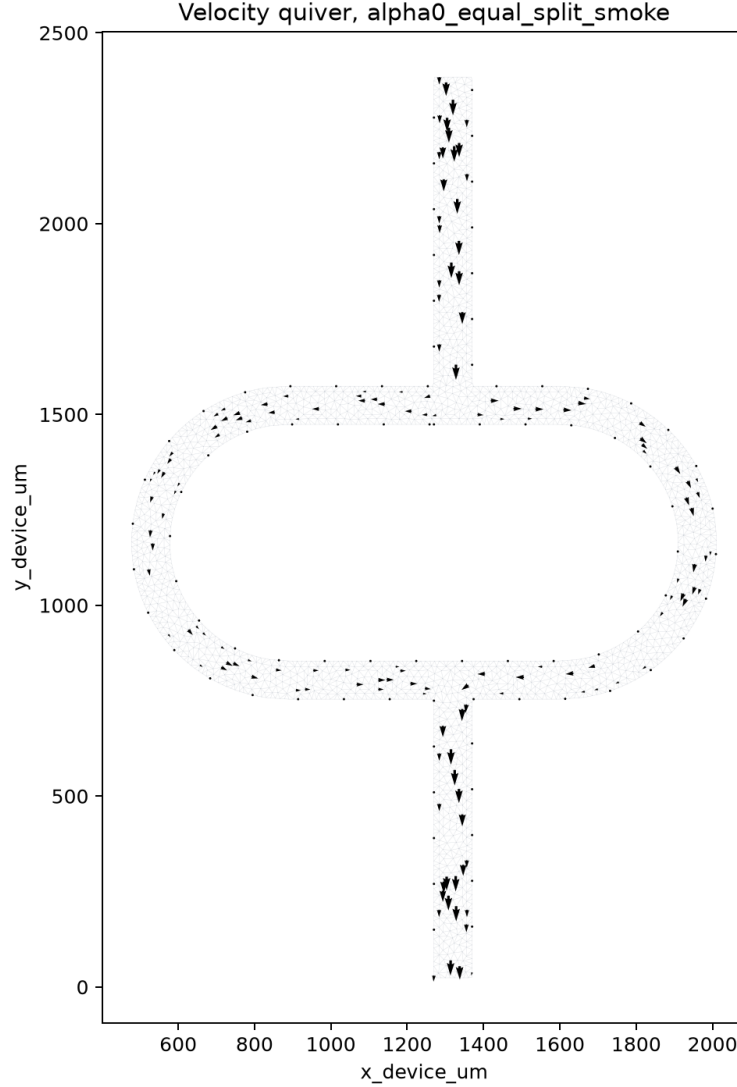


Figure 1: Full-device CFD velocity vectors in device Cartesian coordinates.

5 Physics Geometry: Centerline Module

5.1 Files

- `src/physics/geometry/centerlines.py`
- `src/physics/geometry/types.py`

- `scripts/geometry/build_geometry.py`
- `tests/test_centerline_geometry.py`

5.2 Physical Significance

The centerline is the one-dimensional skeleton of the microfluidic network. It defines the topology, branch lengths, local tangent directions, and normal directions used by later geometry and occupancy calculations. The centerline is not itself a CFD boundary; it is a smooth geometric reference from which channel regions and idealized domains can be constructed.

The current device has four labeled branches:

$$\{\text{inlet}, \text{left}, \text{right}, \text{outlet}\}.$$

The upper junction is the inlet bifurcation into the left and right loop branches. The lower junction merges the two loop branches into the outlet.

5.3 Coordinate and Unit Convention

Raw centerline coordinates are loaded in pixel units. The module immediately stores calibrated arc lengths using:

$$s_\mu = s_{\text{px}} \Delta x, \quad \Delta x = 4 \mu\text{m}/\text{px}.$$

For a branch represented by ordered points $\mathbf{x}_i = (x_i, y_i)$, cumulative arc length is:

$$s_0 = 0, \quad s_i = \sum_{j=1}^i \|\mathbf{x}_j - \mathbf{x}_{j-1}\|_2.$$

The local tangent vector is estimated by centered finite difference at interior points and one-sided difference at endpoints:

$$\mathbf{t}_i = \frac{\mathbf{x}_{i+1} - \mathbf{x}_{i-1}}{\|\mathbf{x}_{i+1} - \mathbf{x}_{i-1}\|_2}.$$

The normal convention is:

$$\mathbf{n}_i = (-t_{y,i}, t_{x,i}).$$

5.4 Implementation Details

The module:

- inspects the centerline CSV schema;
- recognizes branch labels through aliases such as `channel`;
- orders branch points using explicit order columns when available;
- falls back to coordinate-adjacency reconstruction when possible;
- standardizes branch orientation according to physical flow direction;

- validates loop branch lengths against the device YAML;
- saves CSV, JSON, NPZ, and PNG geometry artifacts.

The current centerline file contains labels in the `channel` column. When nearest-neighbor path reconstruction fails for complex branch geometry, the CSV row order is retained as the implicit point sequence. This is documented in the runtime output and does not modify the source centerline.

5.5 Outputs

```
outputs/geometry/asymmetric_loop_h100/centerline_geometry.csv
outputs/geometry/asymmetric_loop_h100/geometry_summary.json
outputs/geometry/asymmetric_loop_h100/device_geometry.npz
outputs/geometry/asymmetric_loop_h100/centerline_geometry.png
```

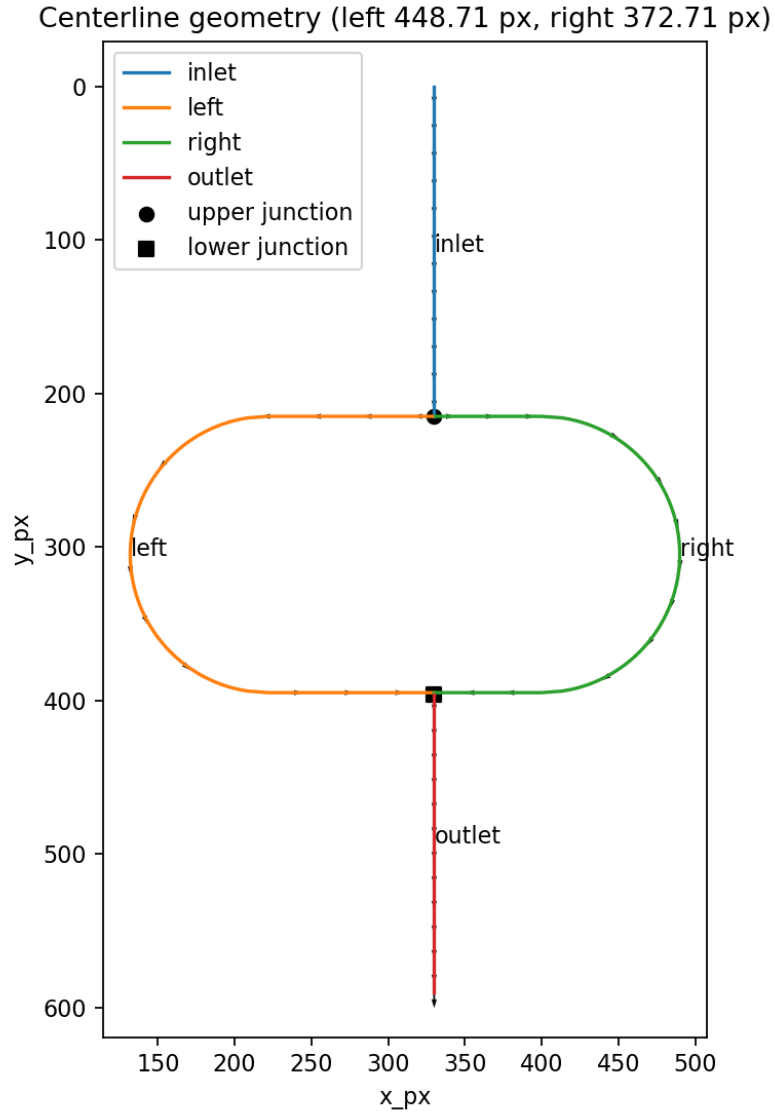


Figure 2: Calibrated centerline geometry diagnostic. The plotted branches encode the physical inlet, left branch, right branch, and outlet topology.

5.6 Execution

```

.\.venv\Scripts\python.exe -m scripts.geometry.build_geometry \
  --experiment configs\experiments\video_2.yml \
  --overwrite

```

6 Physics Geometry: Device Region Labeling

6.1 Files

- src/physics/geometry/regions.py
- scripts/geometry/build.device_regions.py

- `tests/test_device_regions.py`

6.2 Physical Significance

Device region labeling converts the calibrated geometry into a discrete physical partition of the channel. The label map is device-level: it is defined once for the chip geometry and then reused for every droplet frame. This prevents the occupancy module from re-inventing spatial regions on the fly.

The labels are:

ID	Physical region
0	background/unassigned
1	inlet
2	outlet
3	left branch
4	right branch
5	upper junction
6	lower junction

6.3 Governing Geometry

The physical width of the channel is:

$$W = 100 \mu\text{m} = 25 \text{ px.}$$

The region label map is constructed by assigning pixels to the closest centerline branch or configured junction region while respecting the fixed channel geometry. The region labels serve as the spatial basis functions for occupancy:

$$\Omega = \Omega_{\text{inlet}} \cup \Omega_{\text{outlet}} \cup \Omega_{\text{left}} \cup \Omega_{\text{right}} \cup \Omega_{\text{upper junction}} \cup \Omega_{\text{lower junction}}.$$

6.4 Implementation Details

The label builder:

- reads the device geometry and calibration;
- uses preconfigured junction centers and physical junction sizes;
- builds integer labels with no nonzero labels outside the channel;
- validates that all physical regions are nonempty;
- validates connectivity between branches and junctions;
- creates a region overlay for manual inspection.

6.5 Outputs

data/geometry/asymmetric_loop_h100/region_labels.npy
data/geometry/asymmetric_loop_h100/region_metadata.json
data/geometry/asymmetric_loop_h100/region_overlay.png

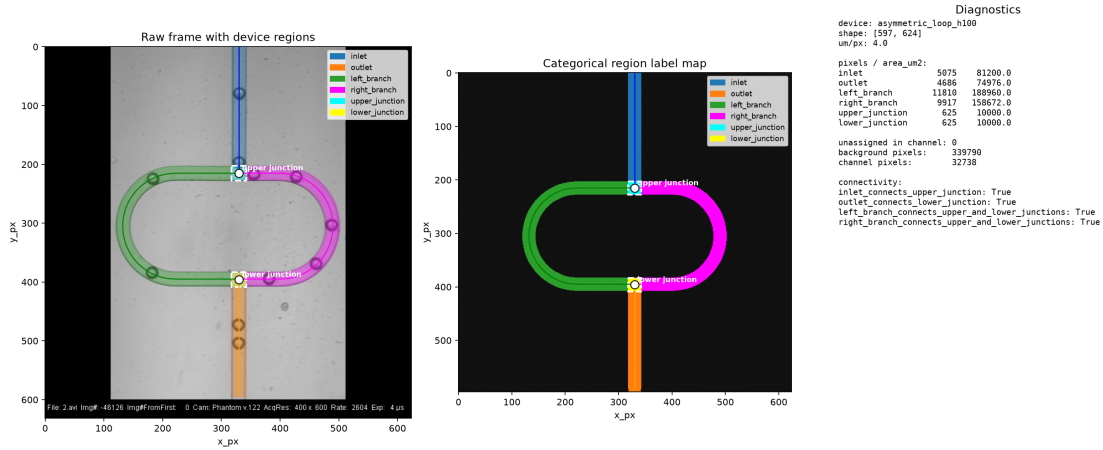


Figure 3: Device-level physical region overlay. This image shows the precomputed region labels used by droplet occupancy. Regions are not recomputed by the occupancy or validation visualizers.

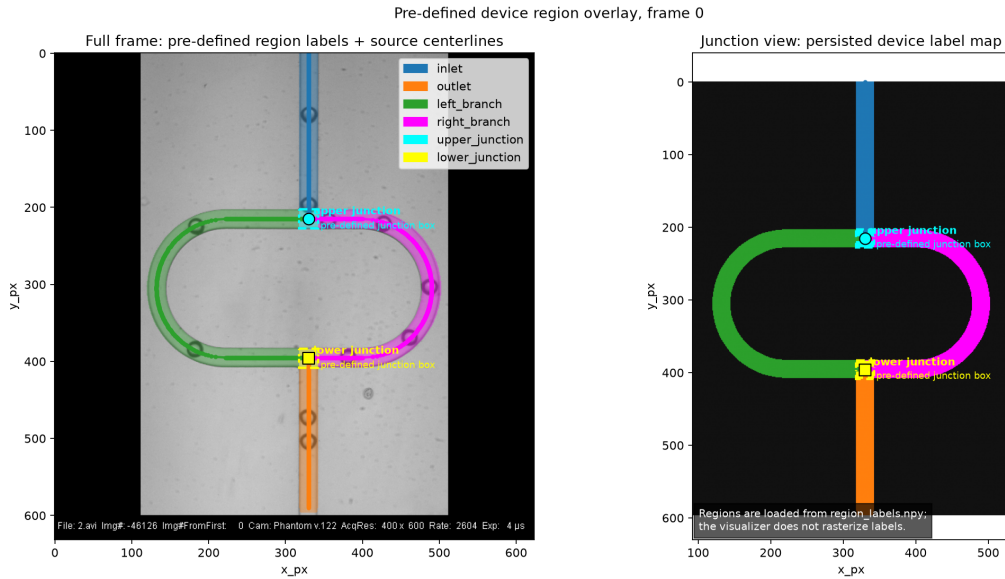


Figure 4: Historical centerline-region diagnostic for frame 0. It is retained as a development figure; the production occupancy path uses the device-level labels in Figure 3.

6.6 Execution

```
.\.venv\Scripts\python.exe -m scripts.geometry.build_device_regions \
--experiment configs\experiments\video_2.yml \
```



```
--frame-index 0 \
--overwrite
```

7 Physics Occupancy: Droplet Fractional Occupancy

7.1 Files

- `src/physics/occupancy/ellipse.py`
- `src/physics/occupancy/calculator.py`
- `scripts/occupancy/compute_droplet_occupancy.py`
- `tests/test_droplet_occupancy.py`

7.2 Physical Significance

Droplet occupancy converts each tracked droplet into a fractional physical state over the device regions. This is the bridge from image-space droplet tracking to physics-aware branch loading. A droplet that is fully inside the left branch should contribute approximately one unit of left-branch occupancy, whereas a droplet spanning a junction or transition contributes fractionally across regions.

7.3 Droplet Footprint Model

Each detected droplet is approximated by an axis-aligned ellipse fitted to its tracking bounding box. If the tracked centroid is (x_c, y_c) , with bounding box width b_x and height b_y , the ellipse is:

$$\left(\frac{x - x_c}{b_x/2}\right)^2 + \left(\frac{y - y_c}{b_y/2}\right)^2 \leq 1.$$

The physical footprint area reported from the bounding box is:

$$A_{\text{ellipse}} = \pi \left(\frac{b_x \Delta x}{2}\right) \left(\frac{b_y \Delta x}{2}\right),$$

where $\Delta x = 4 \mu\text{m}/\text{px}$.

7.4 Raw and Normalized Occupancy

For droplet footprint D and physical region Ω_k , raw occupancy is the fraction of rasterized droplet pixels lying in that region:

$$w_{k,\text{raw}} = \frac{|D \cap \Omega_k|}{|D|}.$$

The physical coverage is:

$$c = \sum_k w_{k,\text{raw}}.$$

When $c > 0$, the normalized occupancy is:

$$w_k = \frac{w_{k,\text{raw}}}{c}.$$

The six-region normalized vector order is:

$$\mathbf{w} = [w_{\text{inlet}}, w_{\text{outlet}}, w_{\text{left}}, w_{\text{right}}, w_{\text{upper junction}}, w_{\text{lower junction}}].$$

7.5 Implementation Details

The module:

- validates the integer region-label map;
- rasterizes the local droplet ellipse without scanning the full image;
- computes raw occupancy for all six physical regions;
- normalizes occupancy whenever physical coverage is nonzero;
- preserves raw coverage diagnostics rather than discarding low-coverage samples;
- records boundary clipping and dominant region.

Low physical coverage is diagnostic, not a reason to zero the normalized vector. Samples with no physical coverage remain noncomputable.

7.6 Outputs

```
outputs/physics/video_2/droplet_occupancy.csv
outputs/physics/video_2/droplet_occupancy_summary.json
outputs/physics/video_2/occupancy_diagnostics.png
outputs/physics/video_2/occupancy_spot_checks.png
outputs/physics/video_2/invalid_occupancy_random_10.png
```

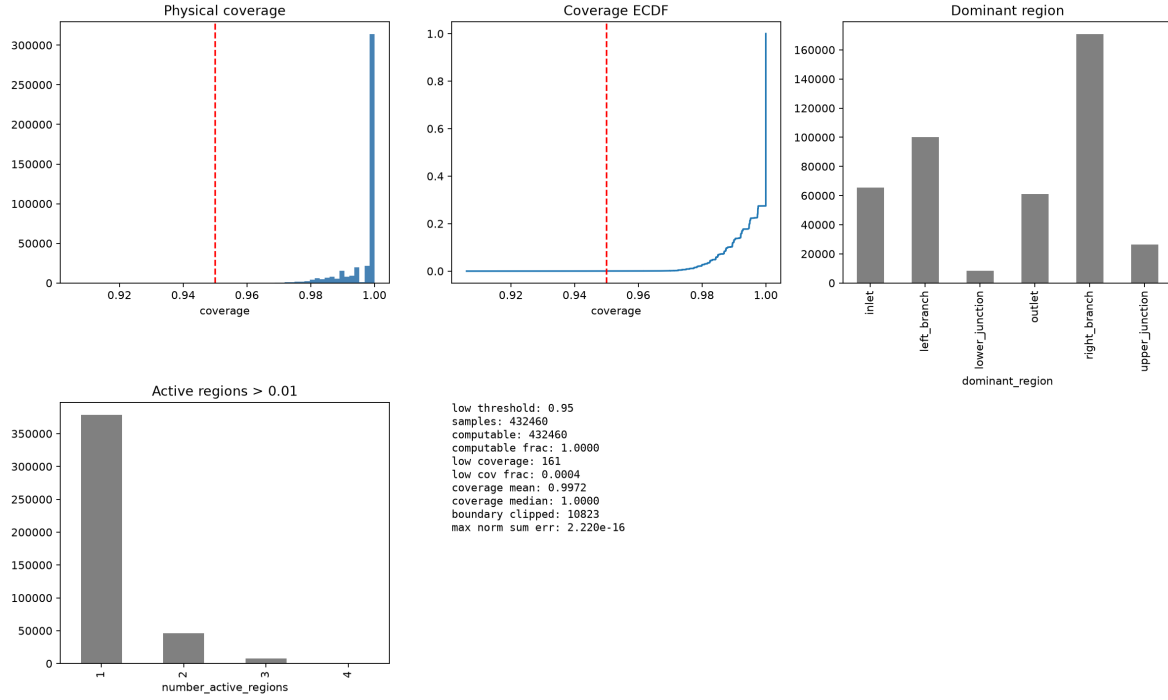


Figure 5: Occupancy diagnostics summarizing physical coverage, active regions, boundary clipping, and normalized occupancy behavior.

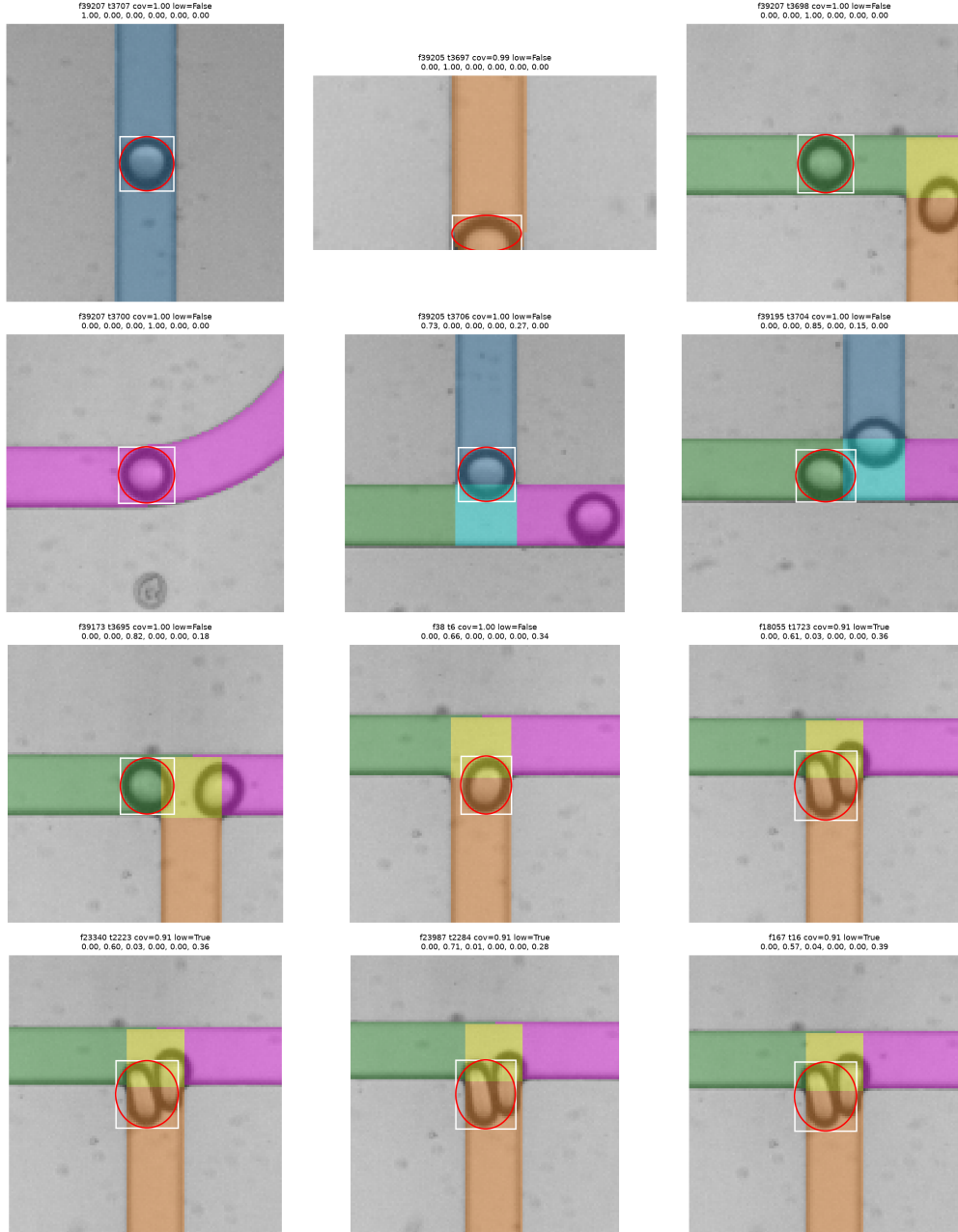


Figure 6: Representative droplet occupancy spot checks. These verify ellipse rasterization against the precomputed device region labels.

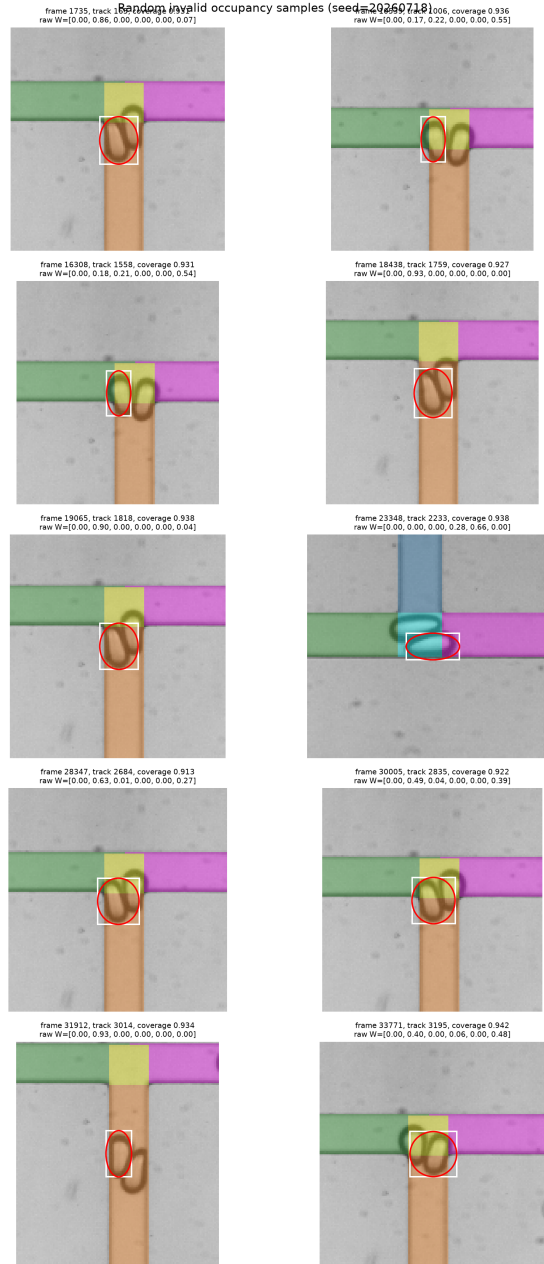


Figure 7: Manual inspection montage for low-coverage or previously invalid occupancy cases.

7.7 Execution

```

.\.venv\Scripts\python.exe -m scripts.occupancy.compute_droplet_occupancy \
--experiment configs\experiments\video_2.yml \
--overwrite

```

8 Physics Hydraulics: Baseline Branch Model

8.1 Files

- `src/physics/hydraulics/baseline.py`
- `scripts/physics/compute_baseline_hydraulics.py`
- `tests/test_baseline_hydraulics.py`

8.2 Physical Significance

This module translates droplet occupancy into a frame-level hydraulic state for the two loop branches. It is a baseline lumped-resistance model, not a full fluid solver. The purpose is to compute physically interpretable branch flow rates and superficial mixture velocities that can later be used as physics features.

The model treats each branch as a hydraulic resistor with resistance proportional to effective branch length:

$$R_i \propto L_{i,\text{eff}}.$$

The baseline does not model junction losses, phase-specific slip, local pressure fields, or droplet–droplet interactions beyond additive resistance.

8.3 Effective Droplet Resistance

An isolated droplet is represented as an equivalent length added to the branch. For the current device:

$$\ell_d = 0.15L_{\text{short}} = 0.15(1491 \mu\text{m}) = 223.65 \mu\text{m}.$$

Effective branch occupancies are sums of normalized droplet branch occupancy:

$$N_{\text{left}}(t) = \sum_j w_{\text{left},j}(t), \quad N_{\text{right}}(t) = \sum_j w_{\text{right},j}(t).$$

The effective branch lengths are:

$$L_{\text{left,eff}}(t) = L_{\text{left}} + \ell_d N_{\text{left}}(t),$$

$$L_{\text{right,eff}}(t) = L_{\text{right}} + \ell_d N_{\text{right}}(t).$$

8.4 Flow Split

The total mixture flow is:

$$Q_{\text{mix}} = Q_{\text{continuous}} + Q_{\text{dispersed}} = 1960 \mu\text{L/hr}.$$

For two parallel hydraulic resistors, the branch flow rates satisfy:

$$Q_{\text{left}} = Q_{\text{mix}} \frac{R_{\text{right}}}{R_{\text{left}} + R_{\text{right}}}, \quad Q_{\text{right}} = Q_{\text{mix}} \frac{R_{\text{left}}}{R_{\text{left}} + R_{\text{right}}}.$$

Since $R_i \propto L_{i,\text{eff}}$, the implementation uses:

$$Q_{\text{left}} = Q_{\text{mix}} \frac{L_{\text{right,eff}}}{L_{\text{left,eff}} + L_{\text{right,eff}}},$$

$$Q_{\text{right}} = Q_{\text{mix}} \frac{L_{\text{left,eff}}}{L_{\text{left,eff}} + L_{\text{right,eff}}}.$$

8.5 Superficial Mixture Velocity

The reported velocities are superficial mixture velocities based on the full channel cross-sectional area:

$$U_i = \frac{Q_i}{A}, \quad A = WH.$$

With $W = H = 100 \mu\text{m}$, the conversion from $\mu\text{L/hr}$ to $\mu\text{m/s}$ is:

$$U_i [\mu\text{m/s}] = \frac{Q_i [\mu\text{L/hr}] \times 10^9 \mu\text{m}^3/\mu\text{L}}{3600 \text{ s/hr} \times WH}.$$

8.6 Implementation Details

The module:

- aggregates occupancy by frame;
- ignores inlet, outlet, and junction occupancy in branch resistance;
- computes effective branch lengths;
- splits total mixture flow between branches;
- converts branch flow to superficial velocity;
- verifies flow conservation for every frame.

The canonical future ML features are:

```
left_velocity_um_s
right_velocity_um_s
```

8.7 Outputs and Diagnostics

```
outputs/physics/video_2/baseline_hydraulic_state.csv
outputs/physics/video_2/baseline_hydraulic_summary.json
outputs/physics/video_2/baseline_hydraulic_diagnostics.png
```

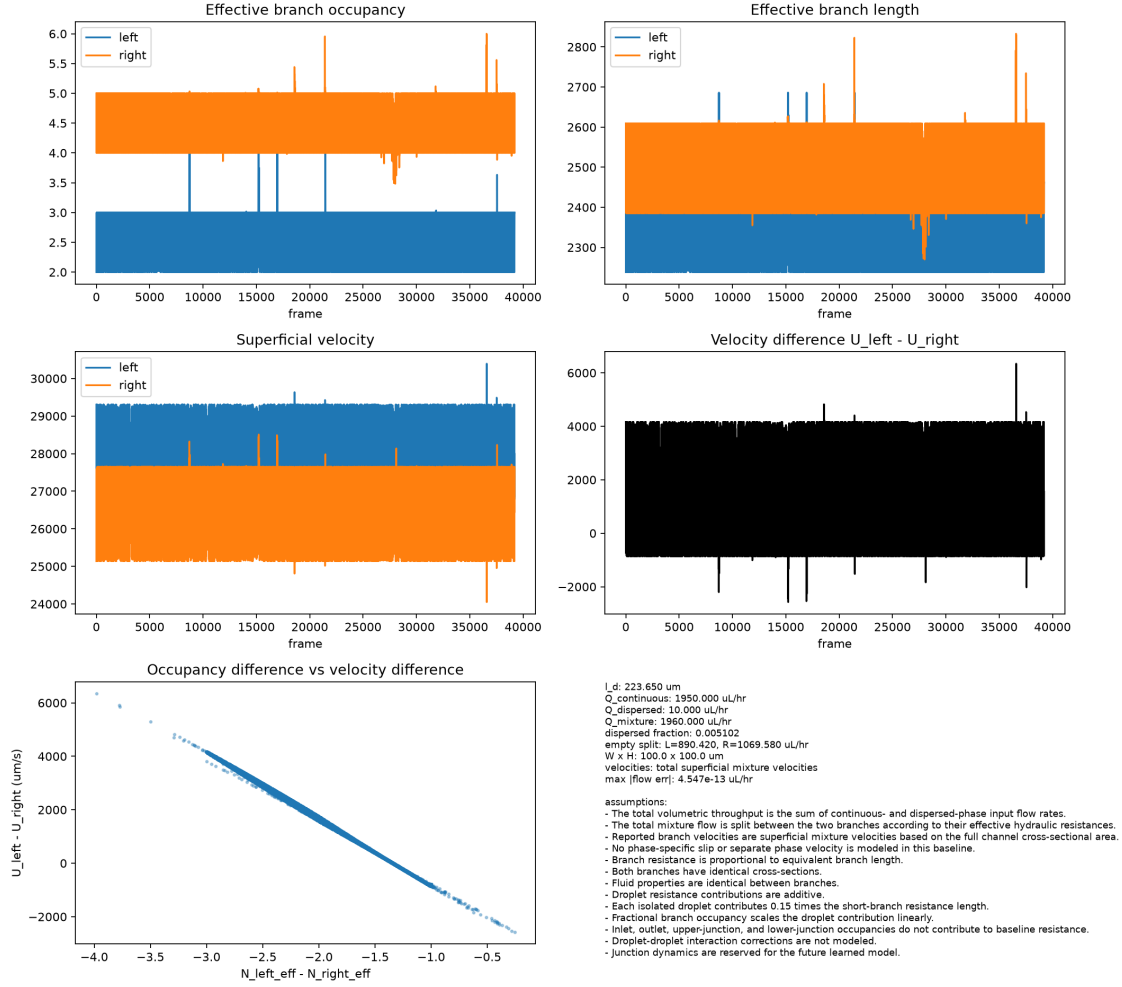


Figure 8: Baseline branch hydraulics diagnostics. The plot shows effective branch occupancy, effective length, superficial velocity, and flow/occupancy relationships.

8.8 Execution

```

.\.venv\Scripts\python.exe -m scripts.physics.compute_baseline_hydraulics \
--experiment configs\experiments\video_2.yml \
--overwrite

```

9 Physics CFD: Full-Device Stokes Model

9.1 Scientific Role

The full-device CFD module provides a spatially resolved, continuum-flow description of the asymmetric loop device. The simulated domain contains the incoming trunk, the upper bifurcation, the long left arm, the short right arm, the lower merge junction, and the downstream outlet. The central solid island is treated as an internal no-slip boundary.

The purpose of this module is to generate a reusable background velocity field for downstream physics enrichment and eventual machine-learning features. The current model is a single-phase,

steady, incompressible Stokes approximation of the continuous phase. Droplets are not resolved as moving deformable bodies in this CFD layer; their influence is represented elsewhere through occupancy and hydraulic-state modules.

centerline geometry $\rightarrow$ Boolean CAD fluid domain $\rightarrow$ triangular finite-element mesh $\rightarrow$ steady Stokes solution $\rightarrow$

9.2 Implemented Files

- `src/physics/full_device_cfd/domain.py`: constructs the full-device CAD polygon from calibrated centerlines.
- `src/physics/full_device_cfd/mesh.py`: generates and labels the constrained triangular finite-element mesh.
- `src/physics/full_device_cfd/solver.py`: assembles and solves the mixed Stokes system on the full-device mesh.
- `src/physics/full_device_cfd/field_verification.py`: compares the production-candidate field against the fine reference field.
- `scripts/physics/build_full_device_mesh.py`: writes the full-device CAD, mesh, and geometry diagnostics.
- `scripts/physics/run_full_device_convergence.py`: runs scalar mesh convergence for the $\alpha = 0$ case.
- `scripts/physics/verify_full_device_field_convergence.py`: performs the final $24\ \mu\text{m}$ versus $12\ \mu\text{m}$ vector-field comparison.

9.3 Configuration

The production full-device CFD configuration is:

`configs/physics/full_device_cfd.yml`

The selected production mesh size is frozen at exactly

$$h_{\text{target}} = 24\ \mu\text{m}, \quad h_{\text{boundary}} = 12\ \mu\text{m}.$$

This value was selected after comparing the complete $24\ \mu\text{m}$ velocity vector field against a $12\ \mu\text{m}$ reference field on a common Cartesian evaluation grid.

9.4 Geometry Construction

The device geometry is reconstructed from the extracted centerlines and fixed channel dimensions, not from a segmentation mask. Coordinates are stored in the device Cartesian frame in micrometers. The channel width and height are both $100\ \mu\text{m}$.

The CAD construction is deliberately geometric:

centerlines $\rightarrow$ offset walls at $\pm 50\ \mu\text{m}$ $\rightarrow$ Boolean union of connected channels $\rightarrow$ central island hole $\rightarrow$ boundary r

This construction yields one connected fluid component with one internal solid island. The inlet and outlet channels are blended into the loop as part of a single continuous CAD domain so that no artificial wall fragments remain inside the bifurcation or merge regions.

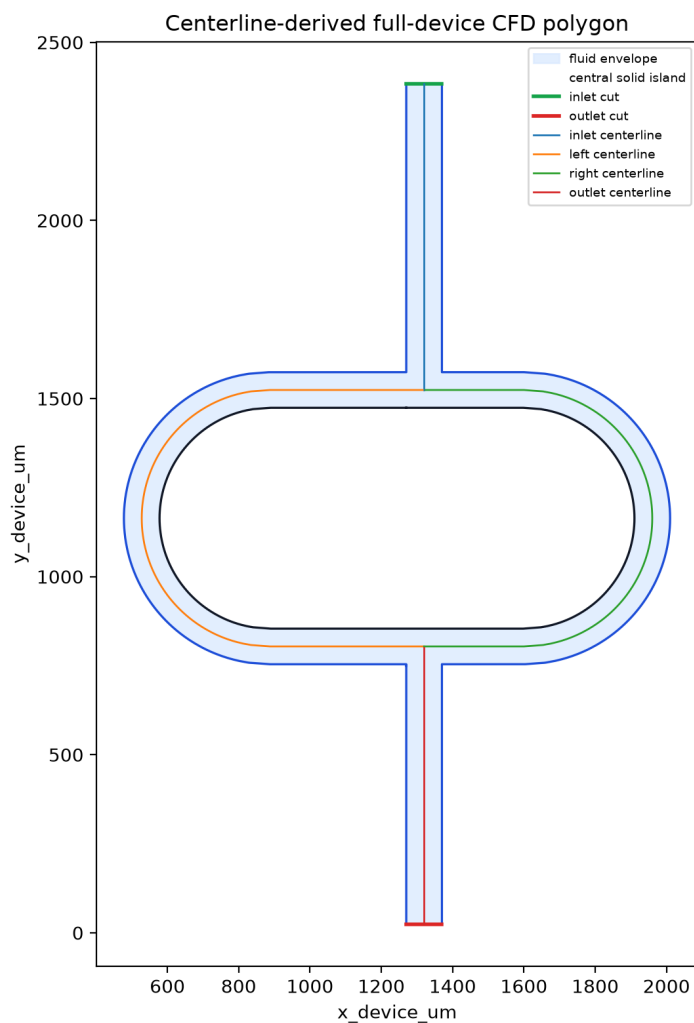


Figure 9: Full-device fluid polygon reconstructed from calibrated centerlines. The blue region is the continuous fluid domain and the white central island is the no-slip solid boundary.

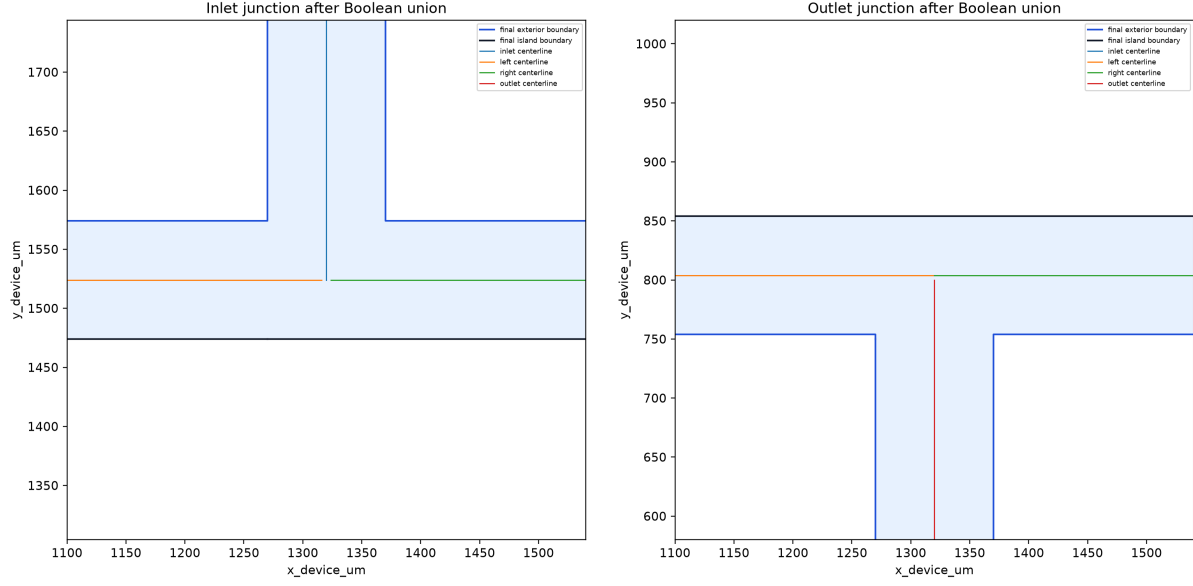


Figure 10: Zoomed CAD diagnostics at the inlet split and outlet merge. These checks verify that the branch transitions are continuous and do not contain internal wall remnants.

9.5 Mesh Generation

The mesher receives the final CAD polygon as a planar straight-line graph. The outer wall and inner island are sampled at the configured boundary spacing, and a constrained triangular mesh is generated over the fluid region. Boundary facets are labeled as inlet, outlet, or wall for finite-element boundary conditions.

For the frozen $24\ \mu\text{m}$ production mesh:

Quantity	Value
Nodes	2339
Elements	3807
Minimum angle	25.727°
Maximum angle	122.392°
Median aspect ratio	1.456
Maximum aspect ratio	2.285
Fluid area	$5.226 \times 10^5\ \mu\text{m}^2$
Boundary length	$1.043 \times 10^4\ \mu\text{m}$

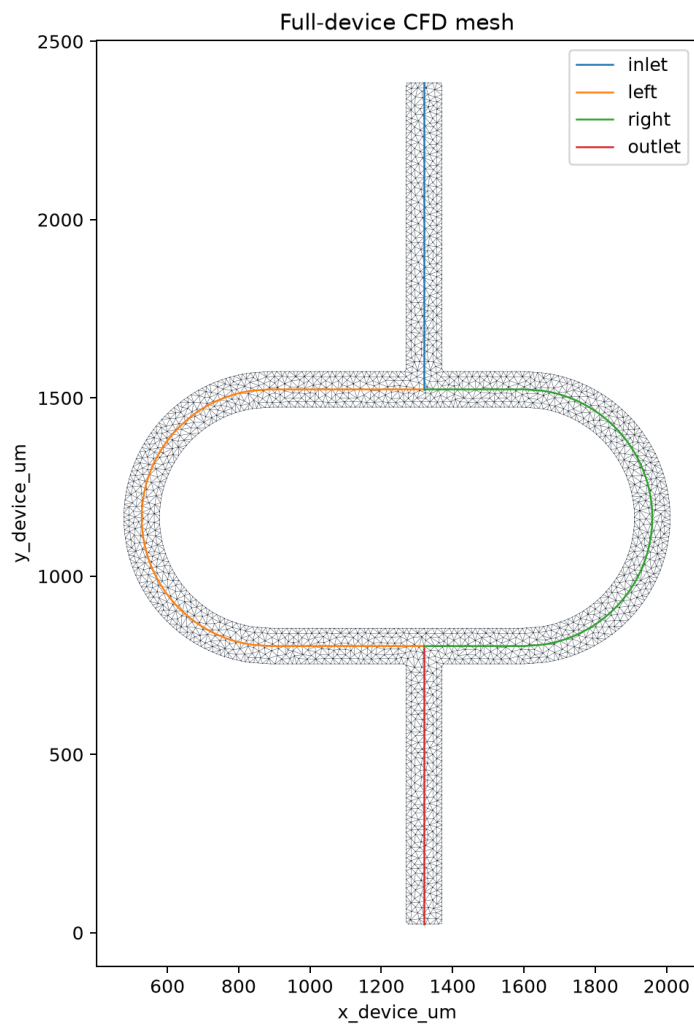


Figure 11: Frozen $24\ \mu\text{m}$ full-device triangular mesh.

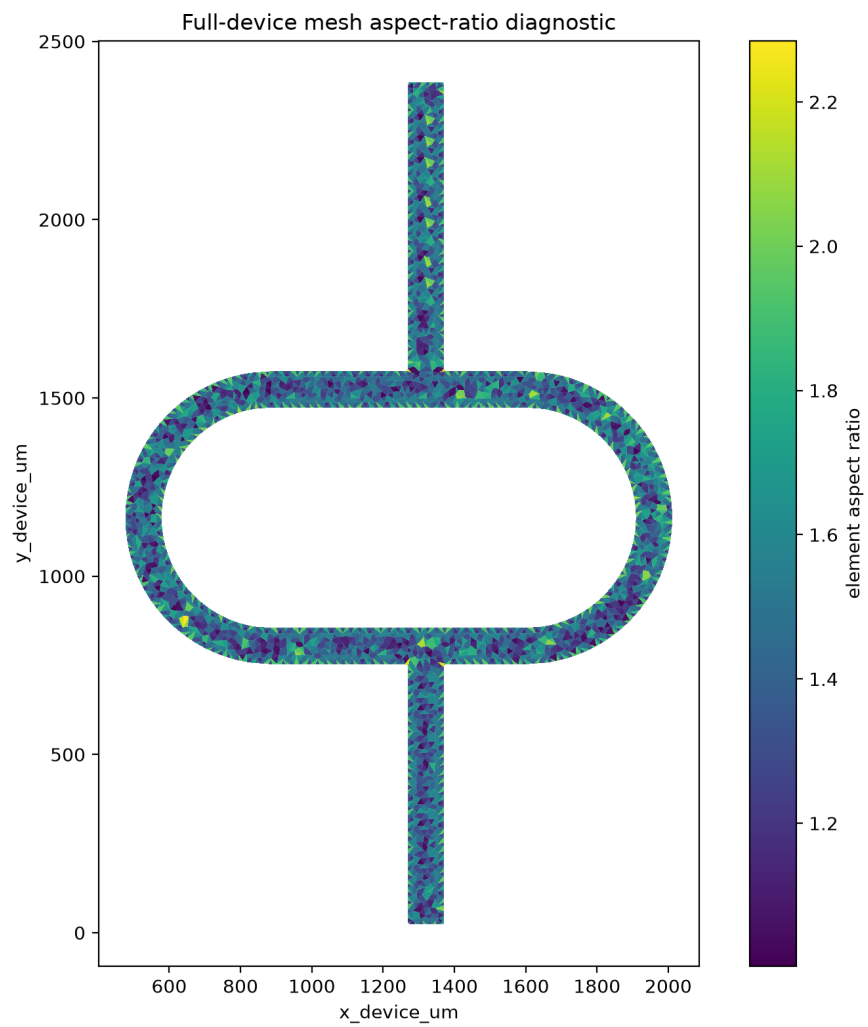


Figure 12: Mesh aspect-ratio diagnostic for the full-device production mesh.

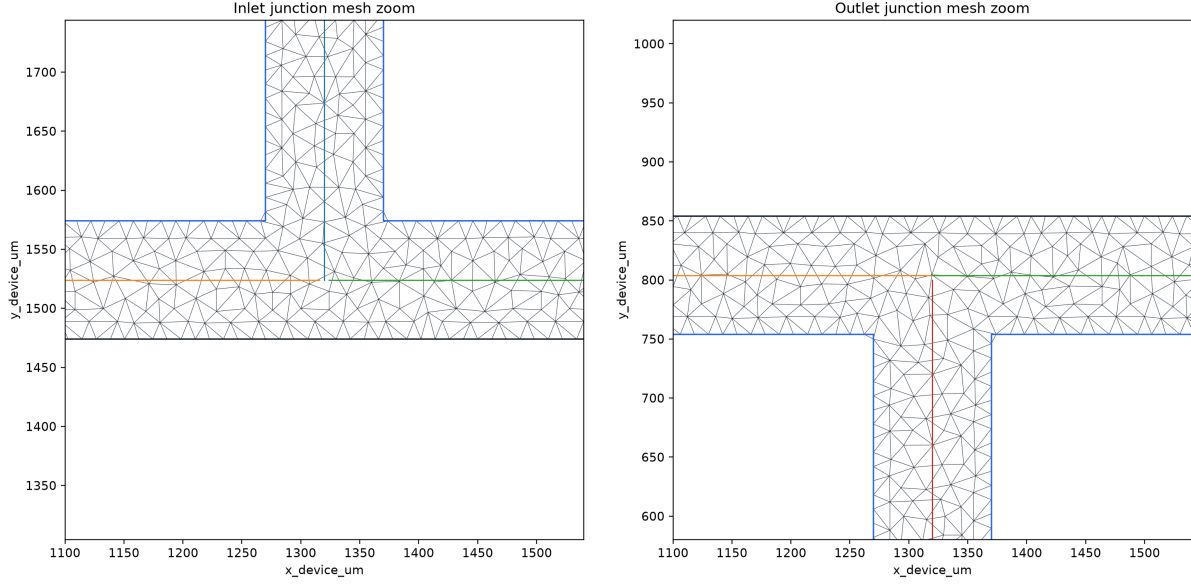


Figure 13: Production mesh near the inlet split and outlet merge.

9.6 Governing Equations

The full-device solver uses the steady incompressible Stokes equations in the two-dimensional channel midplane:

$$-\mu \nabla^2 \mathbf{u} + \nabla p + \alpha(\mathbf{x}) \mathbf{u} = 0, \quad \nabla \cdot \mathbf{u} = 0.$$

Here $\mathbf{u} = (u_x, u_y)$ is the in-plane velocity, p is pressure, μ is dynamic viscosity, and $\alpha(\mathbf{x})$ is an optional distributed branch-resistance field. For the validated $\alpha = 0$ baseline,

$$\alpha_L = \alpha_R = 0,$$

so the equation reduces to the ordinary Stokes problem. The solved two-dimensional velocity is interpreted as a height-averaged background field for the 100 μm -high channel.

9.7 Boundary Conditions

For the $\alpha = 0$ full-device validation case, the boundary formulation is:

- prescribed parabolic inlet velocity profile with the experimental total flow rate;
- no-slip velocity on all channel walls and on the central island;
- natural traction condition at the downstream outlet;
- one pressure gauge degree of freedom to remove the constant pressure null mode;
- no pressure Dirichlet condition on the inlet or outlet.

The imposed inlet flux per unit depth is computed from the experimental total volumetric flow rate Q and channel height H :

$$q_{\text{in}} = \frac{Q}{H}.$$

The mean inlet velocity follows from the channel width W :

$$U_{\text{mean}} = \frac{q_{\text{in}}}{W}.$$

The prescribed planar Poiseuille profile is

$$\mathbf{u}_{\text{in}}(s) = \frac{3}{2}U_{\text{mean}} \left[1 - \left(\frac{2s}{W} \right)^2 \right] \mathbf{t}_{\text{in}}, \quad -\frac{W}{2} \leq s \leq \frac{W}{2},$$

where s is the transverse coordinate and $\mathbf{t}_{\text{in}}$ is the inlet centerline tangent.

9.8 Finite-Element Discretization

The solver uses a Taylor–Hood mixed finite-element pair:

$$\mathbf{u}_h \in [P_2]^2, \quad p_h \in P_1.$$

The weak form is assembled with scikit-fem. Velocity Dirichlet conditions are applied using labeled boundary facets so that P2 edge-midpoint velocity degrees of freedom are retained and constrained correctly. The linear saddle-point system is solved with the direct backend. No pressure-mass stabilization, least-squares fallback, or artificial compressibility term is used.

9.9 Validated $\alpha = 0$ Solution

The first validated full-device solution is saved under:

`outputs/physics/full_device_cfd/alpha0_equal_split_smoke/`

The historical frozen baseline snapshot is:

`outputs/physics/full_device_cfd/baseline_alpha0_v1/`

The $\alpha = 0$ case does not produce a 50/50 split in the asymmetric loop, because the long and short branches have different hydraulic resistances. The computed branch split is:

$$f_L = \frac{Q_L}{Q_L + Q_R} = 0.458119, \quad f_R = 0.541881.$$

Representative scalar quantities for the 24 μm mesh are:

Quantity	Value
Solver backend	scikit-fem/direct
Inlet flux	$-5.4444 \times 10^{-6} \text{ m}^2/\text{s}$
Outlet flux	$5.4774 \times 10^{-6} \text{ m}^2/\text{s}$
Left branch flux	$2.4932 \times 10^{-6} \text{ m}^2/\text{s}$
Right branch flux	$2.9490 \times 10^{-6} \text{ m}^2/\text{s}$
Maximum velocity	$8.1675 \times 10^{-2} \text{ m/s}$
Pressure range	166.436 Pa

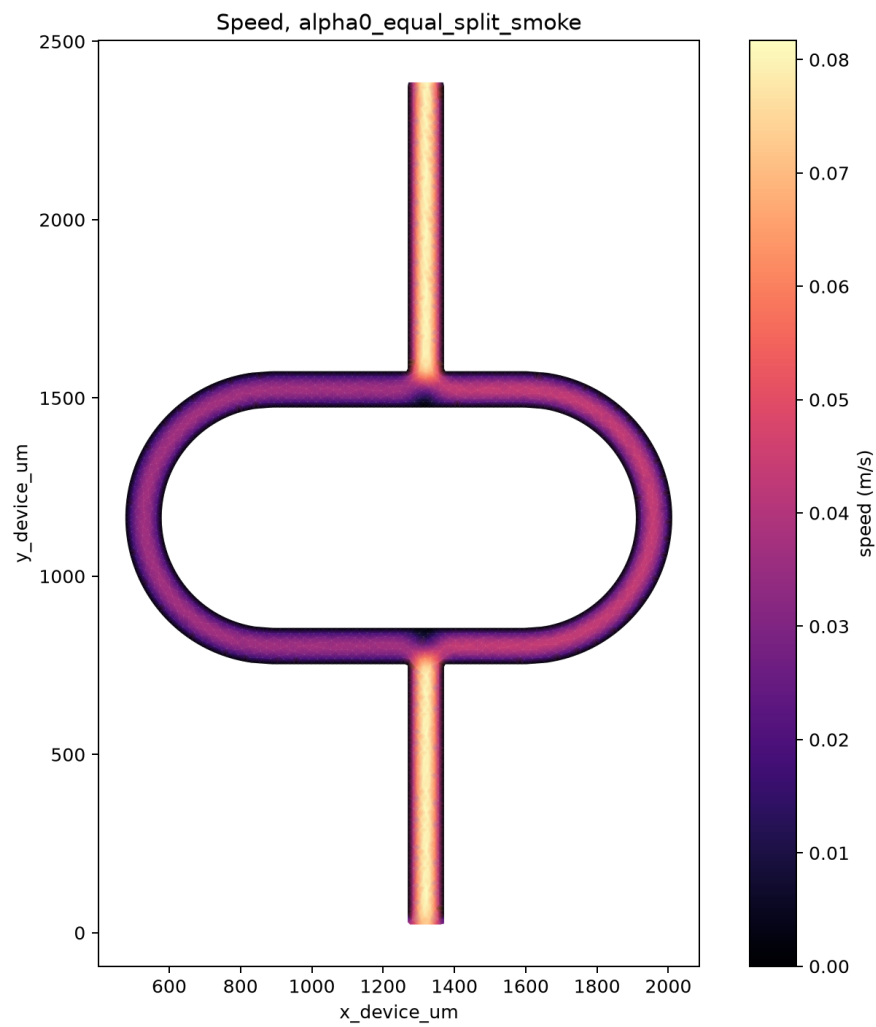


Figure 14: Velocity magnitude for the validated full-device $\alpha = 0$ Stokes solution.

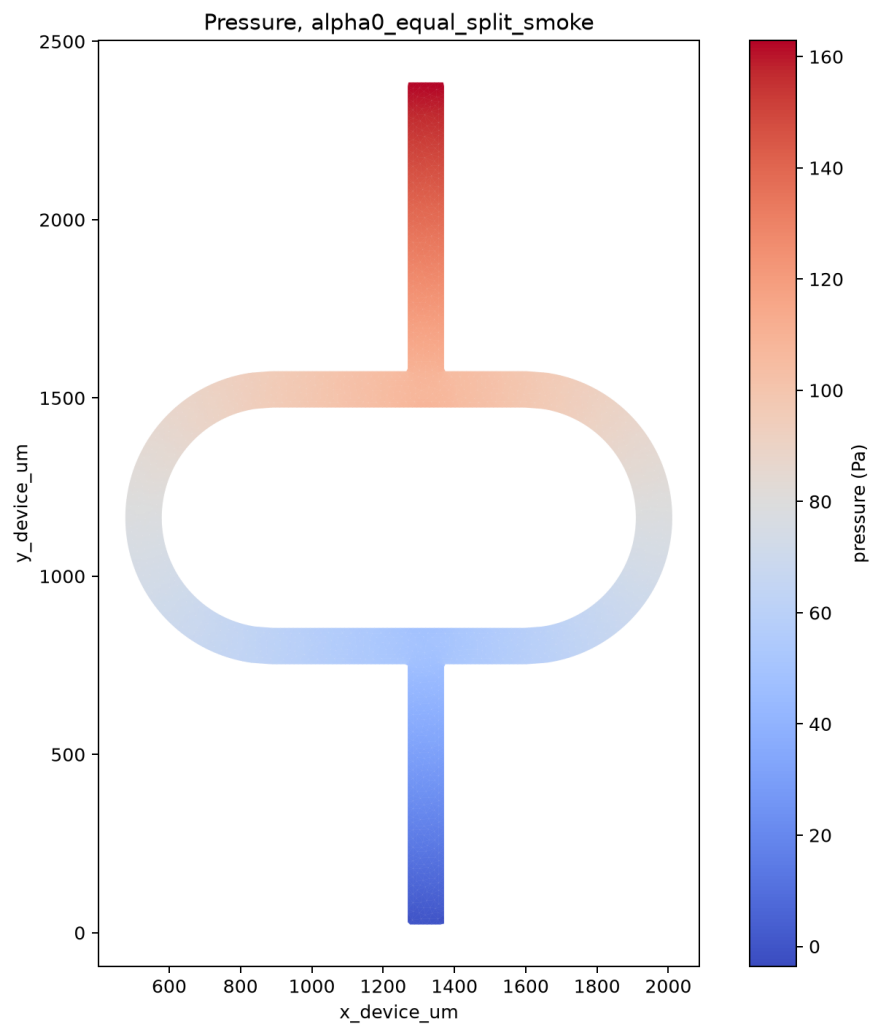


Figure 15: Pressure field for the validated full-device $\alpha = 0$ Stokes solution.

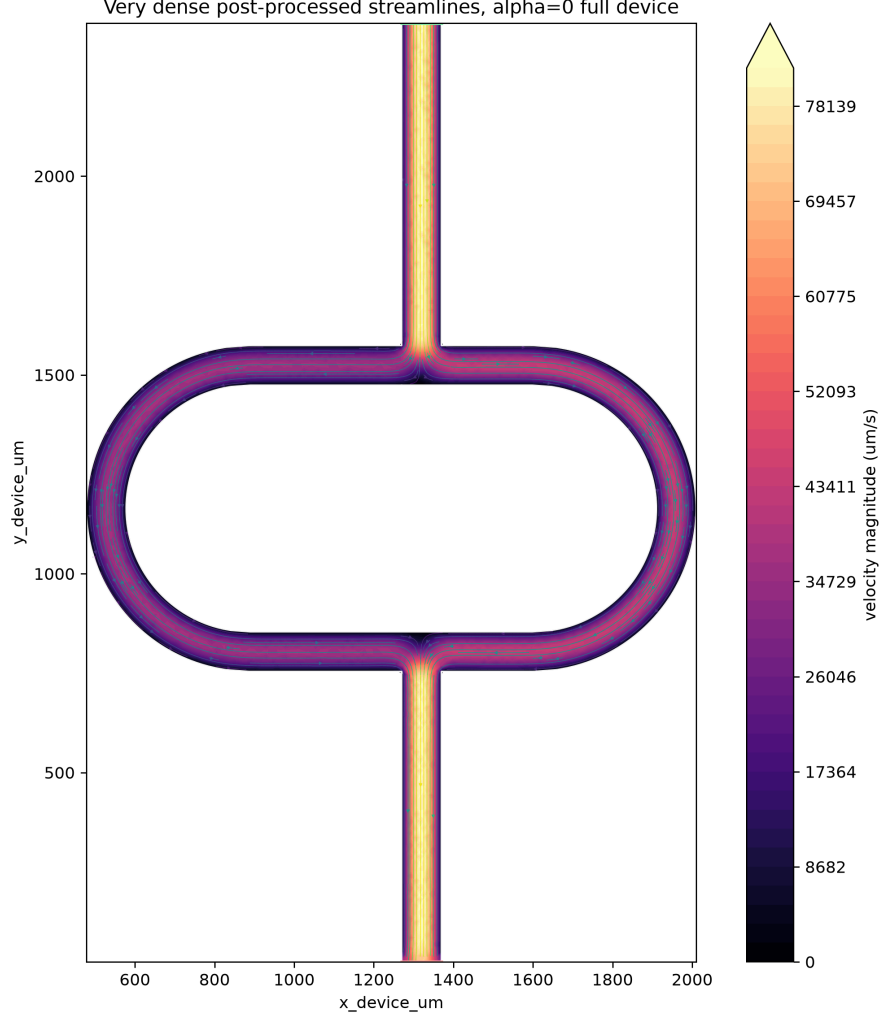


Figure 16: Dense post-processed streamlines sampled from the solved full-device velocity field. These streamlines are visualization samples of a continuous FEM velocity field, not a limit on CFD resolution.

9.10 Scalar Mesh Convergence

A scalar convergence study was run for target element sizes 48, 36, 24, 18, 12 μm . The 12 μm case was used as the fine reference. The branch split and maximum velocity are already stable at 24 μm , while the pressure range continues to move by less than 0.5% relative to the 12 μm reference.

h (μm)	Elements	f_L	$ \Delta f_L $ vs 12	$v_{\max}$ rel. error	Pressure range rel. error
48	1040	0.458190	3.36×10^{-6}	4.32×10^{-4}	1.50×10^{-2}
36	1768	0.456614	1.58×10^{-3}	1.26×10^{-3}	1.28×10^{-2}
24	3807	0.458119	7.41×10^{-5}	8.59×10^{-4}	4.87×10^{-3}
18	6508	0.458378	1.85×10^{-4}	5.82×10^{-4}	2.34×10^{-3}
12	14154	0.458193	reference	reference	reference

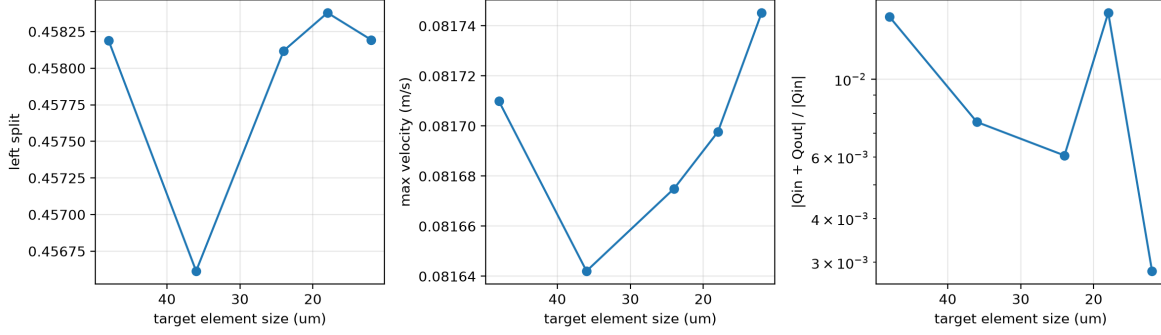


Figure 17: Scalar mesh convergence for the full-device $\alpha = 0$ Stokes solution.

9.11 Field-Level Mesh Verification

The production decision was made using a field-level comparison between the 24 μm candidate and the 12 μm reference. Both solutions were evaluated on the same Cartesian grid with 6 μm spacing. Points outside the final fluid polygon and points inside the central island were masked. Velocity components were interpolated from the FEM basis directly and compared in SI units.

For each region, the vector-field error was computed as

$$E_{\mathbf{u}} = \frac{\|\mathbf{u}_{24} - \mathbf{u}_{12}\|_2}{\|\mathbf{u}_{12}\|_2}.$$

Angular error was computed from

$$\theta = \cos^{-1} \left[\text{clip} \left(\frac{\mathbf{u}_{24} \cdot \mathbf{u}_{12}}{\|\mathbf{u}_{24}\| \|\mathbf{u}_{12}\|}, -1, 1 \right) \right],$$

excluding points with reference speed below 1% of the maximum reference speed. Relative speed errors used the same 1% speed floor to avoid letting stagnation-point noise dominate the decision.

Region	$E_{\mathbf{u}}$	Median angle	95th angle	Speed L2 error
Full domain	0.009808	0.0055°	0.336°	0.008221
Inlet split junction	0.012043	0.0089°	0.829°	0.009419
Outlet merge junction	0.011063	0.0127°	0.865°	0.008380

The inlet separatrix was estimated by tracing the same inlet seed set in both fields. The transition seed location differed by

$$1.533 \mu\text{m} = 1.53\% \text{ of the channel width.}$$

All field-level acceptance criteria passed:

- full-domain vector relative L2 error below 2%;
- inlet-junction vector relative L2 error below 2%;
- outlet-junction vector relative L2 error below 2%;
- median angular error below 1°;

- 95th-percentile angular error below 3° ;
- separatrix-location difference below 2% of channel width.

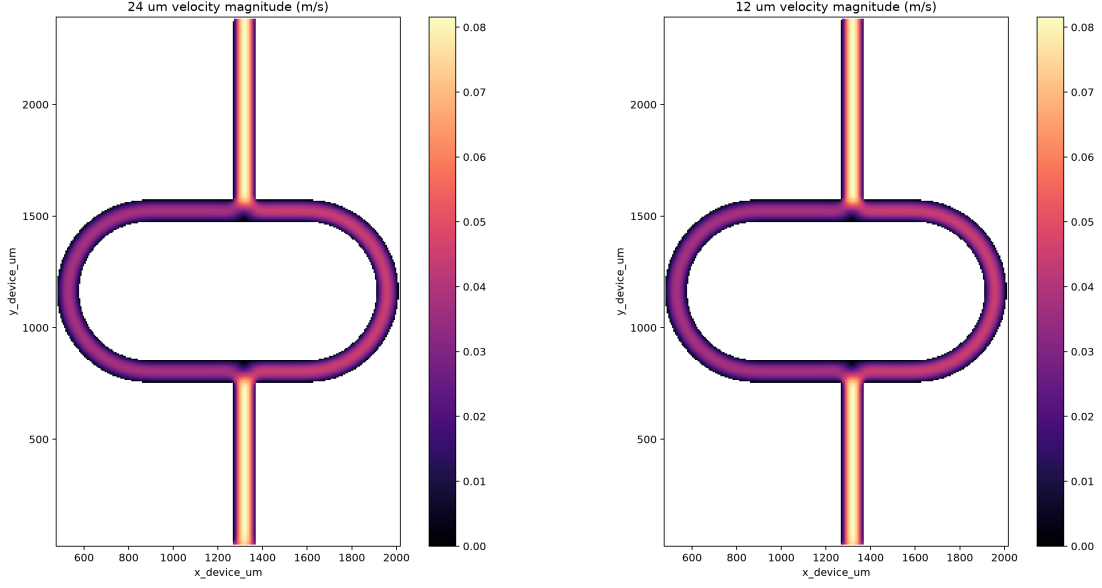


Figure 18: Velocity magnitude on the common grid for the 24 μm candidate and 12 μm reference. Both panels use identical color limits.

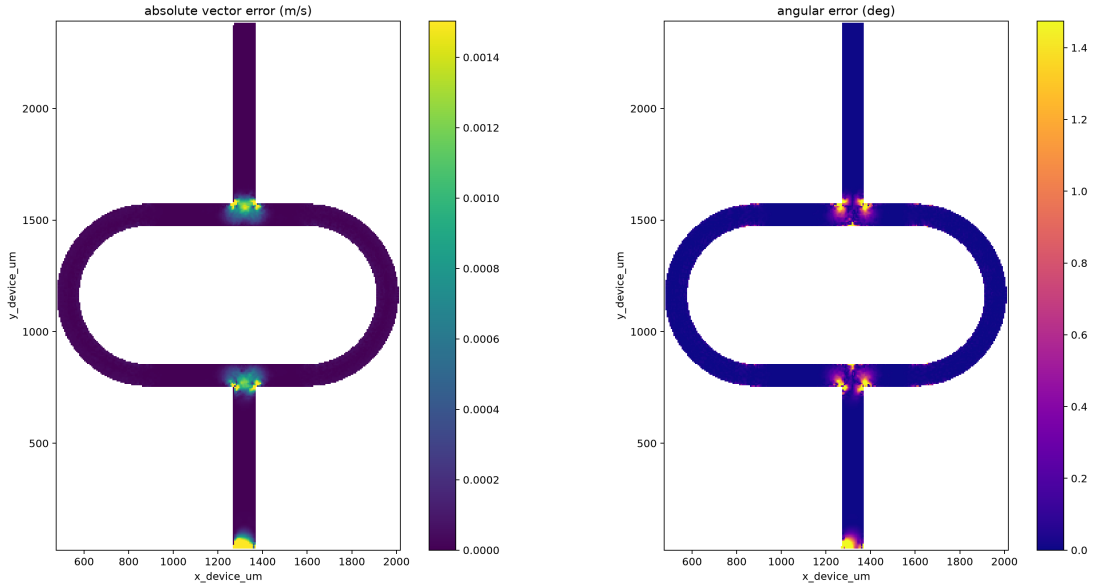


Figure 19: Absolute vector error and angular error between the 24 μm and 12 μm full-device velocity fields.

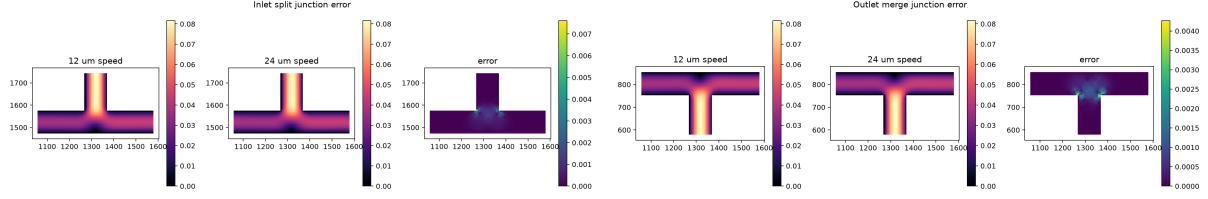


Figure 20: Field-level error zooms at the full-device inlet split and outlet merge. Both fields come from the complete loop geometry.

9.12 Flux Diagnostic

The scalar convergence study showed a non-monotone inlet–outlet flux mismatch. The flux audit confirmed that the current reported boundary fluxes are computed from boundary facets using the FEM P2 velocity field, not from a coarse plotted grid. Each boundary facet is evaluated at its two endpoints and midpoint with Simpson weights.

For the field-verification runs:

Case	Inlet/outlet mismatch	Relative mismatch
24 μm	$3.298 \times 10^{-8} \text{ m}^2/\text{s}$	0.00606
12 μm	$1.541 \times 10^{-8} \text{ m}^2/\text{s}$	0.00283

This scalar mismatch is documented but is not used to reject the 24 μm production mesh, because the field-level errors, branch split, pressure range, and separatrix comparison all satisfy the production criteria.

9.13 Production Decision

The full-device production mesh is frozen at:

$$h_{\text{target}} = 24 \mu\text{m}.$$

The final recommendation is to use the 24 μm mesh as the default full-device CFD mesh. The 12 μm mesh remains the fine reference for verification, not the default production mesh.

9.14 Execution

Build the full-device CAD and production mesh:

```
.\.venv\Scripts\python.exe -m scripts.physics.build_full_device_mesh
```

Run the scalar mesh convergence study:

```
$env:MPLCONFIGDIR='outputs\matplotlib'
.\.venv\Scripts\python.exe scripts\physics\run_full_device_convergence.py '
--target-sizes-um 48 36 24 18 12 '
--output-dir outputs\physics\full_device_cfd\convergence_alpha0_fine
```

Run the final field-level verification:

```
$env:MPLCONFIGDIR='outputs\matplotlib'
.\.venv\Scripts\python.exe scripts\physics\verify_full_device_field_convergence.py '
--grid-spacing-um 6 '
--output-dir outputs\physics\full_device_cfd\convergence\field_comparison_24_vs_12
```

The field-verification artifacts are saved in:

```
outputs/physics/full_device_cfd/convergence/field_comparison_24_vs_12/
```

10 Physics Interpolation: Full-Device Velocity Fields

10.1 Architectural Role

The interpolation module is a downstream consumer of full-device CFD velocity fields. It does not generate geometry, regenerate meshes, solve Stokes systems, or alter CFD outputs. The scientific workflow is:

Hydraulics $\rightarrow \alpha(t)$, Full-device CFD $\rightarrow \mathbf{u}(x, y; \alpha_i)$, Interpolation $\rightarrow \mathbf{u}(x, y; \alpha)$.

Here α is the left-branch flow fraction,

$$\alpha = \frac{Q_L}{Q_{\text{in}}}, \quad Q_R = (1 - \alpha)Q_{\text{in}}.$$

The present implementation interprets the two-dimensional CFD field as a height-averaged background-flow approximation over the full loop domain. Future ML modules can sample this continuous background field, but no transformer integration is implemented in this module.

10.2 Files

- `src/physics/interpolation/library.py`: discovers and validates the frozen CFD split library.
- `src/physics/interpolation/split_interpolator.py`: chooses neighboring split cases and linearly interpolates P2 velocity coefficients.
- `src/physics/interpolation/field_sampler.py`: evaluates an interpolated finite-element velocity field at arbitrary physical coordinates.
- `src/physics/interpolation/types.py`: typed data structures for loaded cases, interpolated fields, and sampled values.
- `src/physics/interpolation/validation.py`: withheld-case validation and scientific consistency checks.
- `scripts/physics/interpolation/validate_velocity_interpolation.py`: command-line validation entry point.

10.3 Interpolation Model

The frozen CFD library contains discrete solutions at

$$\alpha_i \in \{0.05, 0.10, \dots, 0.95\}.$$

For a requested α , the module finds neighboring stored fields $\alpha_{\text{low}} \leq \alpha \leq \alpha_{\text{high}}$ and computes

$$w = \frac{\alpha - \alpha_{\text{low}}}{\alpha_{\text{high}} - \alpha_{\text{low}}}.$$

Because all Version 1 CFD cases use the same production mesh, the same P2 velocity basis, and the same DOF ordering, interpolation is performed directly on the finite-element velocity coefficients:

$$\mathbf{U}(\alpha) = (1 - w)\mathbf{U}(\alpha_{\text{low}}) + w\mathbf{U}(\alpha_{\text{high}}).$$

This is coefficient interpolation, not interpolation of rendered figures, streamlines, pressure plots, or regular-grid visualizations. If α matches a stored library fraction to the documented tolerance, the stored field is returned exactly and no blend is performed.

10.4 Sampling

The public sampling operations name the coordinate frame explicitly:

```
library = VelocityFieldLibrary.from_directory(
    "outputs/physics/full_device_cfd/solutions"
)
field = library.interpolate(left_fraction=0.33)
samples_cfd = field.sample_cfd(points_cfd_um)
samples_device = field.sample_device(points_device_um, coordinate_convention)
```

`sample_cfd` is the low-level native operation. It expects points in the frozen CFD frame $(x_{\text{cfd}}, y_{\text{cfd}})$, where $+y$ points downward, and returns velocity components in the same frame. `sample_device` expects points in the device Cartesian frame, where $+y$ points upward, and returns device-frame velocity components. The older `sample(points_um)` method is retained only as a backward-compatible alias for `sample_cfd`.

The sampler reconstructs the scikit-fem P2 vector basis on the shared production mesh and evaluates the FEM field directly:

$$\mathbf{u}(\mathbf{x}; \alpha) = \sum_j U_j(\alpha) \phi_j(\mathbf{x}).$$

The output velocity units are meters per second in the requested output frame. Points outside the CFD domain are flagged with `inside_domain = false`; their velocity, speed, and direction values are returned as NaN. For near-zero-speed points inside the domain, the velocity components are retained while the unit direction is NaN.

10.5 Validation

The validation routine withholds stored CFD cases at $\alpha = 0.20, 0.40, 0.60, 0.80$. Each withheld field is reconstructed from its two neighboring cases, then compared against the direct CFD result at the shared velocity DOFs and at a common set of physical sample points inside the junction domain.

α	neighbors	w	DOF RMSE	sample speed RMSE	mean rel. error	mass residual
0.20	0.15/0.25	0.500	2.491×10^{-15}	1.853×10^{-15}	5.998×10^{-14}	5.487×10^{-17}
0.40	0.35/0.45	0.500	1.951×10^{-15}	1.955×10^{-15}	4.282×10^{-14}	6.754×10^{-18}
0.60	0.55/0.65	0.500	2.042×10^{-15}	2.017×10^{-15}	3.656×10^{-14}	-4.136×10^{-17}
0.80	0.75/0.85	0.500	1.936×10^{-15}	1.878×10^{-15}	6.156×10^{-14}	-8.948×10^{-17}

These near-machine-precision errors are expected because the current prescribed split Stokes formulation is linear in the imposed outlet flux split on a fixed mesh. The validation nevertheless remains important because it protects the library contract: mesh identity, DOF ordering, field units, exact-match behavior, and output immutability.

Scientific consistency checks confirmed:

- no NaN values inside the valid sampled domain;
- maximum mass-balance residual $1.135 \times 10^{-16} \text{ m}^2/\text{s}$;
- inlet profile unchanged across tested α ;
- maximum velocity on labeled no-slip wall facets equal to 0.0 m/s;
- linear interpolation preserves the discrete incompressibility diagnostic to the same scale as the stored CFD solutions.

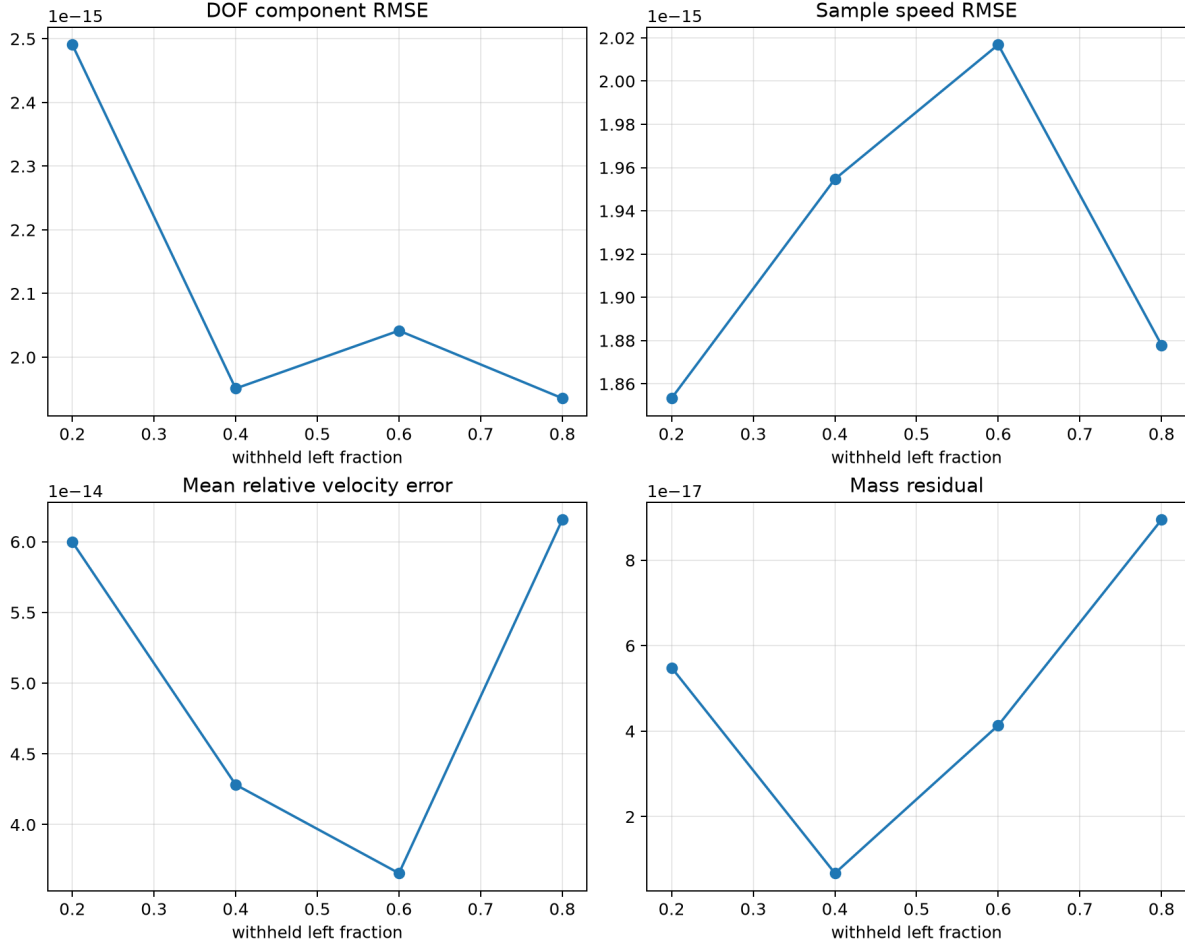


Figure 21: Withheld-case interpolation errors versus left-flow fraction. The errors are computed from FEM velocity coefficients and direct physical-domain samples, not from plotted images.

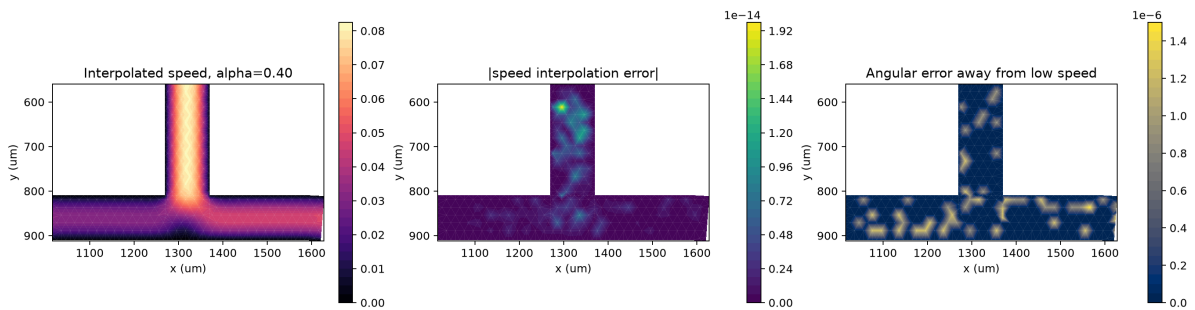


Figure 22: Representative withheld-case validation for $\alpha = 0.40$: interpolated speed, speed error, and angular error away from low-speed regions.

10.6 Execution

```
.\.venv\Scripts\python.exe -m scripts.physics.interpolation.validate_velocity_interpolation --
```

The validation outputs are written outside the frozen CFD library:

`outputs/physics/interpolation/validation/`

11 Physics Enrichment: Tracking Table With Hydraulic and CFD Context

11.1 Purpose

The physics-enrichment layer constructs a downstream feature superset of the tracked droplet table. It combines immutable droplet tracking, framewise baseline hydraulics, and sampled CFD background velocity from the full-device CFD interpolation library:

$$\text{tracked droplets} + \text{hydraulic state}(t) + \mathbf{u}_{\text{CFD}}(x, y; \alpha(t)) \rightarrow \text{physics-enriched tracking}.$$

The enriched tracking table is not itself a canonical ML dataset. Canonical dataset builders will later select controlled state-variable subsets from this single enriched source.

11.2 Immutable Inputs

The module treats the following files as read-only upstream artifacts:

- `data/processed/2/tracked_features.csv`
- `outputs/physics/video.2/baseline_hydraulic_state.csv`
- `outputs/physics/full_device_cfd/solutions/`

No tracking file, hydraulic file, CFD solution, mesh, or interpolation-library artifact is regenerated or modified by this layer.

11.3 Files

- `src/physics/enrichment/tracking_enricher.py`: orchestrates the join, coordinate mapping, CFD sampling, validation, summaries, and diagnostics.
- `src/physics/enrichment/coordinate_mapping.py`: explicit pixel-to-physical-coordinate transform.
- `src/physics/enrichment/validation.py`: row preservation, hydraulic join, field-sampling, and output-integrity checks.
- `src/physics/enrichment/types.py`: enrichment configuration and summary dataclasses.
- `scripts/physics/enrichment/build_physics_enriched_tracking.py`: command-line entry point.

11.4 Hydraulic-State Join

The tracked table is joined to the hydraulic state using the frame index:

$$\text{join key} = \text{frame}.$$

The join is validated as many-to-one: every tracked droplet row must map to exactly one hydraulic state row, and the hydraulic table must contain no duplicate frame entries. The framewise flow fractions are derived from the validated baseline hydraulic outputs:

$$\alpha_L(t) = \frac{Q_L(t)}{Q_{\text{mix}}(t)}, \quad \alpha_R(t) = \frac{Q_R(t)}{Q_{\text{mix}}(t)}.$$

The output stores these as `left_flow_fraction` and `right_flow_fraction`. The branch superficial velocities are converted from $\mu\text{m/s}$ to m/s . The inlet superficial velocity is computed from the total mixture flow rate and the $100 \mu\text{m} \times 100 \mu\text{m}$ channel cross-section.

11.5 Coordinate Mapping

The tracking centroids are image-pixel coordinates. The enrichment layer stores device Cartesian coordinates in micrometers. The full-device CFD geometry uses the same physical device frame, so no crop translation or rotation is required:

$$x_{\text{device},\mu\text{m}} = x_{\text{px}}\Delta_{\mu\text{m}/\text{px}}, \quad y_{\text{device},\mu\text{m}} = (y_{\text{reference},\text{px}} - y_{\text{px}})\Delta_{\mu\text{m}/\text{px}},$$

The same convention is used when sampling the full-device CFD field:

$$x_{\text{CFD},\mu\text{m}} = x_{\text{device},\mu\text{m}}, \quad y_{\text{CFD},\mu\text{m}} = y_{\text{device},\mu\text{m}}.$$

11.6 CFD Background Sampling

For each row, the framewise hydraulic split $\alpha_L(t)$ selects a continuous background velocity field from the interpolation module:

$$\mathbf{u}_{\text{bg}}(x, y, t) = \mathbf{u}_{\text{CFD}}(x_{\text{CFD}}, y_{\text{CFD}}; \alpha_L(t)).$$

The implementation samples the same P2 finite-element fields used by the interpolation module. For efficiency, stored CFD cases are evaluated at all inside-domain droplet positions and blended row-wise. This is equivalent to sampling the interpolated FEM coefficient field because basis evaluation is linear:

$$\sum_j \left[(1-w)U_j^a + wU_j^b \right] \phi_j(\mathbf{x}) = (1-w) \sum_j U_j^a \phi_j(\mathbf{x}) + w \sum_j U_j^b \phi_j(\mathbf{x}).$$

11.7 Output Schema

The enriched table preserves every original tracking column and appends:

- coordinates: `x_device_um`, `y_device_um`, `x_cfd_um`, `y_cfd_um`;
- hydraulic state: `left_flow_fraction`, `right_flow_fraction`, `inlet_superficial_velocity_m_per_s`, `left_branch_superficial_velocity_m_per_s`, `right_branch_superficial_velocity_m_per_s`;

- CFD background: `background_u_x_device_m_per_s`, `background_u_y_device_m_per_s`, `background_speed_m_per_s`, `background_direction_x`, `background_direction_y`, `inside_cfd_domain`;
- interpolation provenance: `cf_d_left_fraction`, `cf_d_right_fraction`, `cf_d_alpha_low`, `cf_d_alpha_high`, `cf_d_interpolation_weight`, `cf_d_exact_match`;
- reproducibility/version columns: `physics_enrichment_version` and `coordinate_transform_version`.

If available, `dominant_region` from the occupancy table is retained as a diagnostic aid. Centered finite-difference observed velocity directions are also added in the device Cartesian frame for flow-direction alignment diagnostics.

11.8 Missing-Field Semantics

The full-device CFD field covers the loop geometry. Droplets outside the final fluid polygon are marked with

```
inside_cfd_domain = false
```

and their background velocity, speed, and direction fields are NaN. These values are not replaced by zero because zero denotes a physical stagnant velocity, whereas NaN denotes unavailable CFD coverage.

11.9 Validation Results

The most recent full `video_2` enrichment artifact contains 432,460 rows and 44 columns. That full table was not regenerated during the coordinate convention audit; the audit standardizes the code-level schema used by the next full enrichment build. All 15 original tracking columns are preserved at the front of the table.

Quantity	Value
Rows inside CFD domain	88,995
Inside-domain fraction	20.58%
Unique tracks entering CFD domain	3,609
Rows outside CFD domain with NaN background fields	343,465
Hydraulic join coverage	100%

Inside-domain rows by device region:

Region	Rows
inlet	19,777
left branch	14,140
right branch	28,525
upper junction	26,553

Flow-direction alignment compares centered finite-difference droplet direction against sampled background-fluid direction. This diagnostic is used to check coordinate orientation and gross physical consistency; imperfect alignment is not automatically a bug because droplets are finite, confined, interacting objects.

Metric	Value
Valid comparisons	88,983
Mean cosine similarity	0.944611
Median cosine similarity	0.999846
Mean angular difference	9.651°
Median angular difference	1.005°

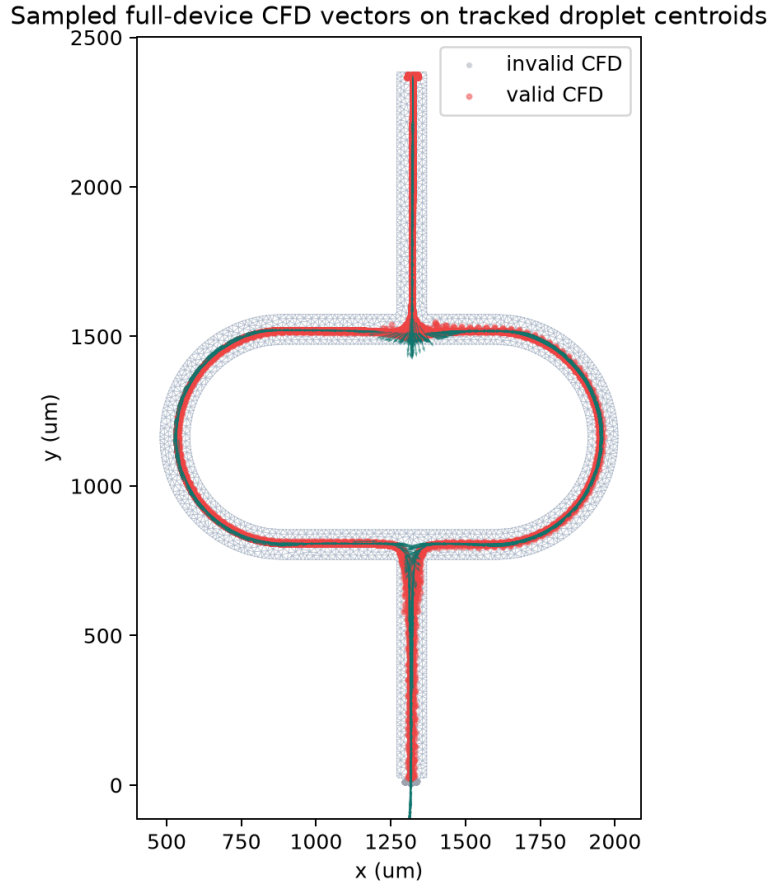


Figure 23: Representative tracked-droplet overlay on the CFD junction mesh. Inside-domain droplets are distinguished from outside-domain droplets, and sampled background velocity vectors are shown for covered droplets.

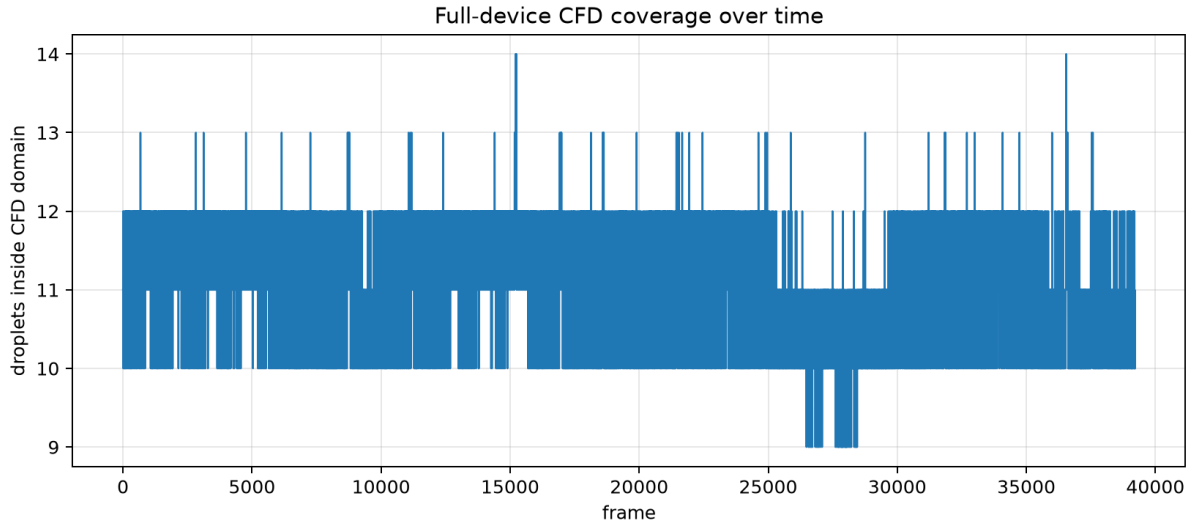


Figure 24: Number of tracked droplets inside the CFD domain as a function of frame.

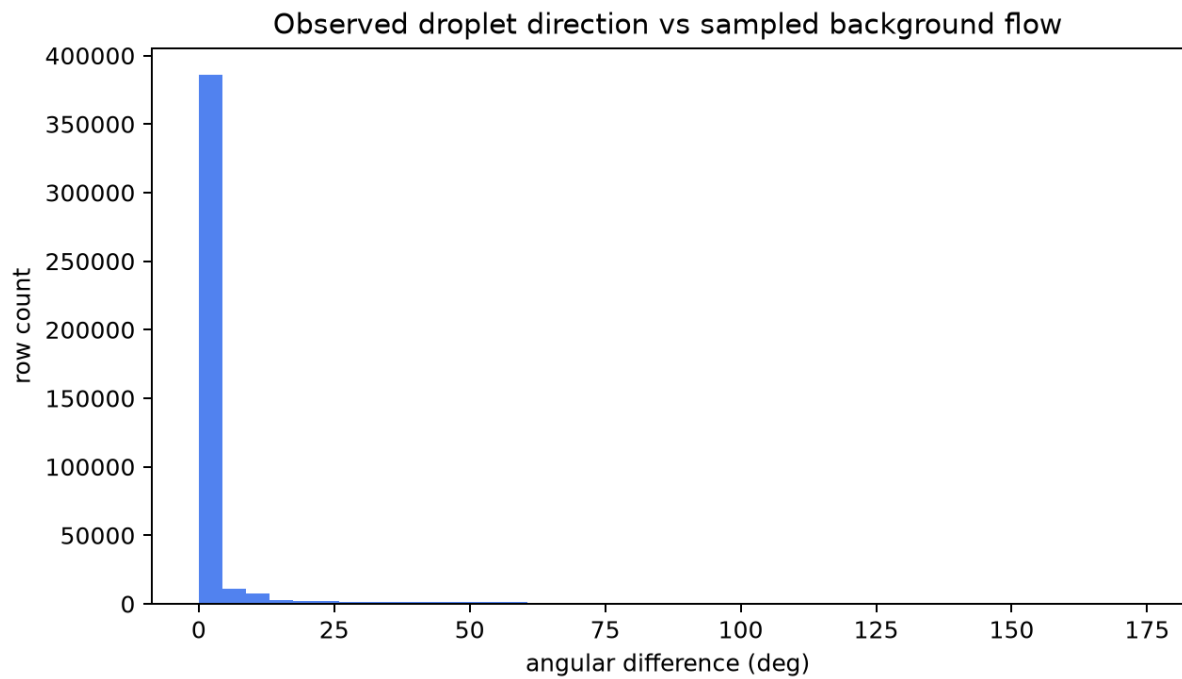


Figure 25: Angular difference between observed droplet motion direction and sampled background-flow direction for droplets inside the CFD patch.

11.10 Execution

```
.\.venv\Scripts\python.exe -m scripts.physics.enrichment.build_physics_enriched_tracking --exp
```

Outputs are written to:

outputs/physics/video_2/enrichment/

12 Physics State Placeholder

12.1 Files

- `src/physics/state/physics_state.py`
- `src/physics/state/__init__.py`

12.2 Future Scientific Role

The physics-state package is a placeholder for a unified interface between the physics engine and the ML models. In future phases it should expose synchronized state variables such as geometry coordinates, droplet occupancy, branch hydraulic features, CFD velocity lookups, and possibly streamline-derived features.

No functionality is implemented in this package yet. This is intentional: the architecture reserves a stable location without prematurely defining an API.

13 Physics-Embedded Markovian Rollout Model

13.1 Scientific Purpose

The physics-embedded Markovian model tests whether a droplet’s instantaneous state, augmented by Eulerian continuous-phase CFD information and reduced hydraulic state, is sufficient to predict near-future droplet motion. The model is Markovian in the sense that it consumes only the current frame, $T_{\text{history}} = 1$, then advances the state autoregressively.

13.2 State Definition

The canonical training input is `outputs/processed/2/canonical_dataset_v2/canonical_dataset_v2.npz`. Each droplet state has 15 features:

1. x
2. y
3. v_x
4. v_y
5. circularity
6. `cfid_u`
7. `cfid_v`
8. left-flow fraction
9. occupancy in inlet channel
10. occupancy in inlet junction
11. occupancy in left branch
12. occupancy in right branch

13. occupancy in outlet junction
14. occupancy in outlet channel
15. cfd_valid

The network input dimension is therefore $F = 15$. The prediction target is unchanged from the previous Markovian baseline:

$$\hat{\mathbf{y}}_{t+1} = \begin{bmatrix} \hat{v}_{x,t+1} \\ \hat{v}_{y,t+1} \end{bmatrix}.$$

No loss is applied to `cfd_u`, `cfd_v`, flow split, occupancy fractions, or `cfd_valid`. Those quantities are state inputs, not supervised targets.

13.3 Autoregressive Rollout

At rollout step k , the model predicts velocity from the current state:

$$(\hat{v}_{x,k}, \hat{v}_{y,k}) = f_{\theta}(\mathbf{s}_k).$$

Predicted positions are integrated with the same first-order kinematics used by the previous Markovian trainer:

$$\hat{x}_{k+1} = \hat{x}_k + \hat{v}_{x,k}, \quad \hat{y}_{k+1} = \hat{y}_k + \hat{v}_{y,k}.$$

Newly appearing droplets are injected from truth, preserving the previous boundary-conditioned rollout semantics. Non-target observed features (`cfd`, `split`, `occupancy`, and `circularity`) are refreshed from the corresponding observed future row when the droplet is present. This matches the physics-model attention visualization and evaluation code; no future target velocity or future target position is used as model input.

13.4 Loss Masking

The dataset returns both a droplet-presence mask and a CFD-valid supervision mask. The velocity loss is computed only where:

$$\mathcal{M}_{\text{loss}} = \mathcal{M}_{\text{present}} \wedge \mathcal{M}_{\text{cfd valid}} \wedge \neg \mathcal{M}_{\text{truth injection}}.$$

This prevents the cropped downstream outlet region, where CFD coverage is intentionally absent because of the video metadata crop, from contributing to the supervised velocity objective. Invalid CFD values are neutralized numerically only after retaining `cfd_valid` as a feature.

13.5 Implementation

The training entry point is:

```
python -m scripts.training.train_physics_markovian \
    --config configs/experiments/physics_markovian_v1.yml
```

The first production configuration uses:

- $T_{\text{history}} = 1$;

- rollout horizon = 50;
- maximum droplets per window = 64;
- transformer width $d_{\text{model}} = 128$;
- 4 attention heads;
- 4 encoder layers;
- target features (v_x, v_y) .

The final uploaded training checkpoint used in the attention visualizations was `outputs/models/physics_markovian_v1/latest_checkpoint.pt` epoch 50. The best checkpoint copied into the self-contained evaluation bundle is `src/evaluation/models/physics_markovian_v1/best_checkpoint.pt` epoch 33.

13.6 Attention and Trajectory Diagnostics

Attention videos are generated with:

```
python -m scripts.training.visualize_physics_markovian_attention \
    --checkpoint outputs/models/physics_markovian_v1/latest_checkpoint.pt \
    --window-index 125 \
    --target-slot 4 \
    --length 100 \
    --output-dir outputs/models/physics_markovian_v1/attention_junction_stall_track2664_final_epoch50
```

For track 2664, a stall period in the upper junction occurs over rollout steps 21–28. The final epoch-50 diagnostics reported:

- 100 valid rollout rows;
- mean position error 7.33 px;
- maximum position error 9.79 px;
- mean true speed during the stall interval 0.111 px/frame.

14 Validation: Droplet Speed versus Branch Velocity

14.1 Files

- `scripts/validation/validate_droplet_branch_velocity.py`
- `tests/test_droplet_branch_velocity_validation.py`

14.2 Scientific Question

This validation asks whether temporal variations in measured droplet speed co-vary with temporal variations in the calculated baseline branch superficial velocity. It is not a proof that droplet speed equals branch velocity.

The measured droplet speed is derived from tracked centroid positions in **px/frame**. The baseline branch velocity is in $\mu\text{m/s}$. Because the comparison uses per-track standardization, it compares temporal variation, not absolute velocity scale.

14.3 Measured Speed from Centered Difference

For each track, rows are sorted by frame. For interior frames with adjacent observations exactly one frame apart:

$$v_x(t) = \frac{x(t+1) - x(t-1)}{2}, \quad v_y(t) = \frac{y(t+1) - y(t-1)}{2}.$$

The measured speed is:

$$u_{\text{drop}}(t) = \sqrt{v_x(t)^2 + v_y(t)^2}.$$

Endpoints and samples adjacent to tracking gaps are excluded. Track boundaries are never mixed.

14.4 Branch-Interior Filtering

Samples are retained only when a droplet is clearly inside exactly one branch:

$$w_{\text{left}} \geq 0.95 \quad \text{xor} \quad w_{\text{right}} \geq 0.95.$$

For left-branch samples:

$$u_{\text{branch}}(t) = U_{\text{left}}(t).$$

For right-branch samples:

$$u_{\text{branch}}(t) = U_{\text{right}}(t).$$

There is no interpolation between left and right branch velocities.

14.5 Normalization and Correlation

For each selected (`track_id`, `branch`) segment, both signals are standardized independently:

$$z_{\text{drop}}(t) = \frac{u_{\text{drop}}(t) - \bar{u}_{\text{drop}}}{\text{std}(u_{\text{drop}})},$$

$$z_{\text{branch}}(t) = \frac{u_{\text{branch}}(t) - \bar{u}_{\text{branch}}}{\text{std}(u_{\text{branch}})}.$$

The standard deviation uses $\text{ddof} = 1$. Pearson and Spearman correlations are computed at zero lag on the final aligned normalized samples. No smoothing or lag optimization is applied.

14.6 Candidate Selection

The current data contain shorter right-branch interior segments than left-branch segments. The selected cutoff is:

$$n_{\text{min}} = 60.$$

This prevents the entire right branch from being excluded. Candidate ranking is deterministic and based on branch-velocity standard deviation, not correlation. When both branches have enough candidates, the validator selects five left and five right examples.

14.7 Current Results

Metric	Value
Eligible candidates	3595
Eligible left candidates	1148
Eligible right candidates	2447
Selected left candidates	5
Selected right candidates	5
Pooled samples	742
Pooled Pearson r	0.712629
Pooled Spearman ρ	0.718781

Track ID	Branch	Samples	Pearson r	Spearman ρ
3463	left	83	0.603518	0.561827
254	left	83	0.756515	0.846252
3127	left	84	0.803099	0.831943
1713	left	84	0.837714	0.859534
3460	left	75	0.527768	0.493072
3461	right	67	0.668621	0.660548
257	right	66	0.767774	0.779150
3465	right	66	0.811301	0.717638
3462	right	67	0.876987	0.876327
1712	right	67	0.458025	0.526406

14.8 Outputs and Figures

outputs/physics/video_2/droplet_branch_velocity_validation/
 selected_tracks.csv
 pooled_normalized_samples.csv
 droplet_branch_velocity_summary.json
 droplet_branch_velocity_scatter.png
 track_<track_id>_normalized_velocity.png

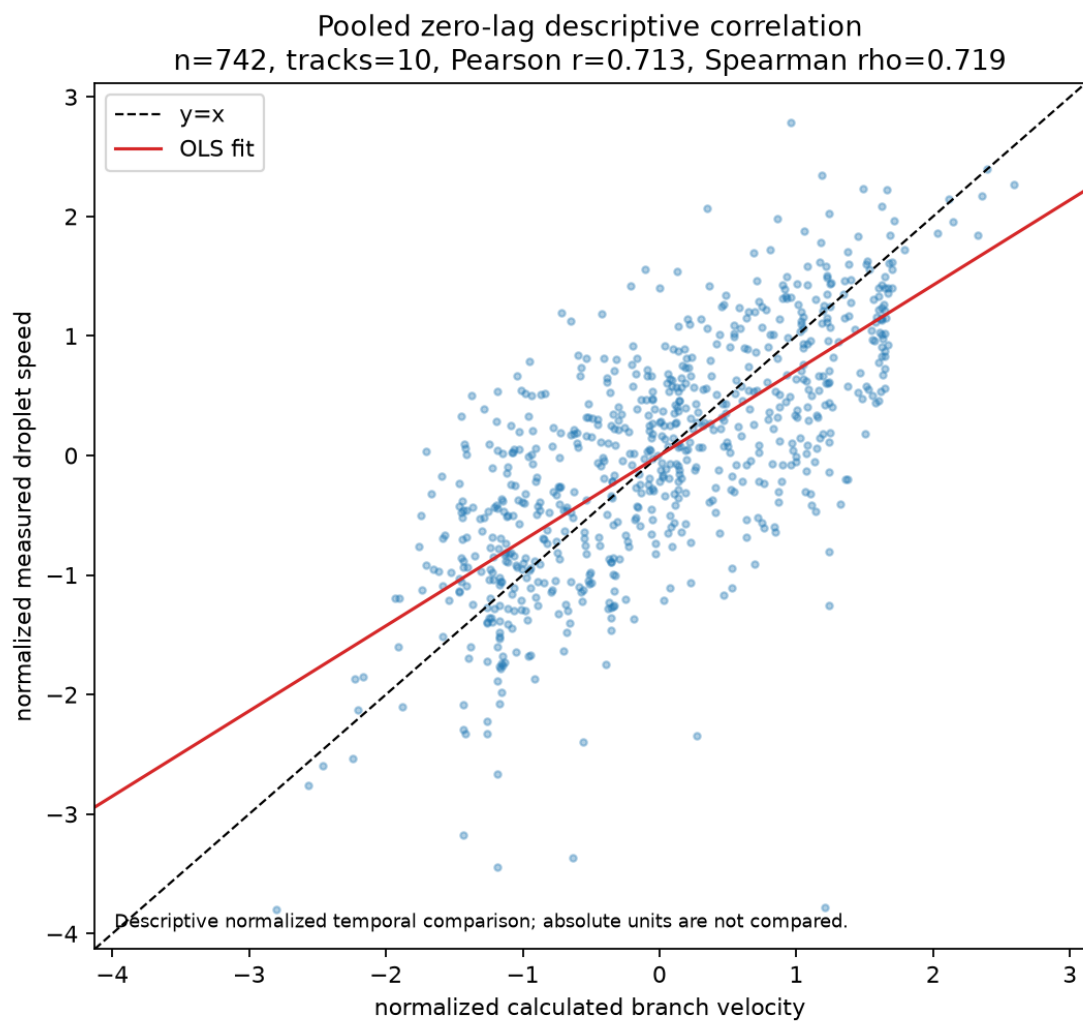


Figure 26: Pooled normalized comparison of calculated branch superficial velocity and measured droplet speed. The pooled correlation is descriptive because repeated frames within a trajectory are temporally correlated.

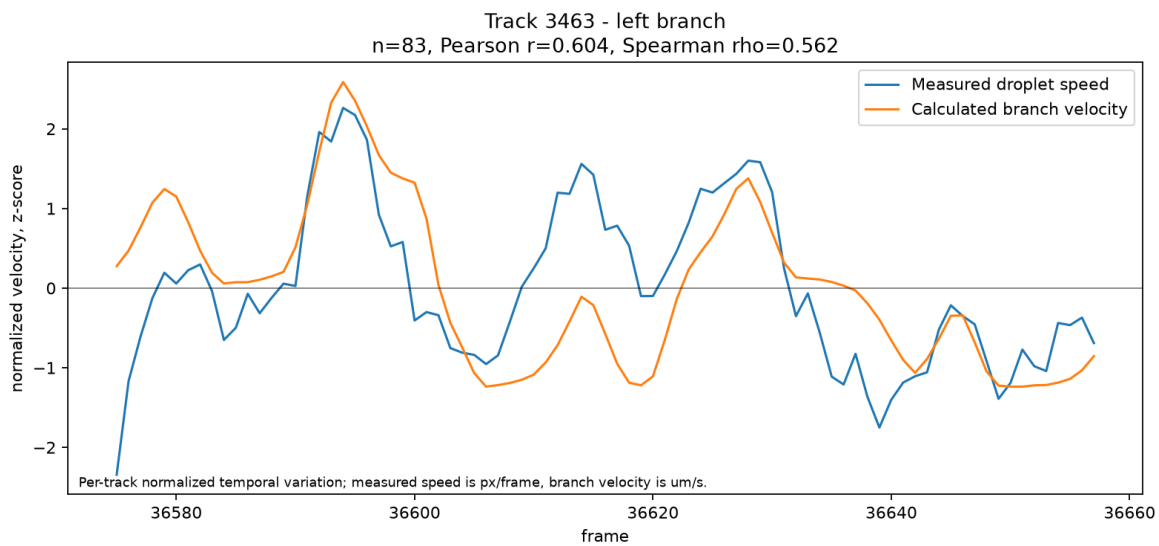


Figure 27: Selected left-branch validation track 3463.

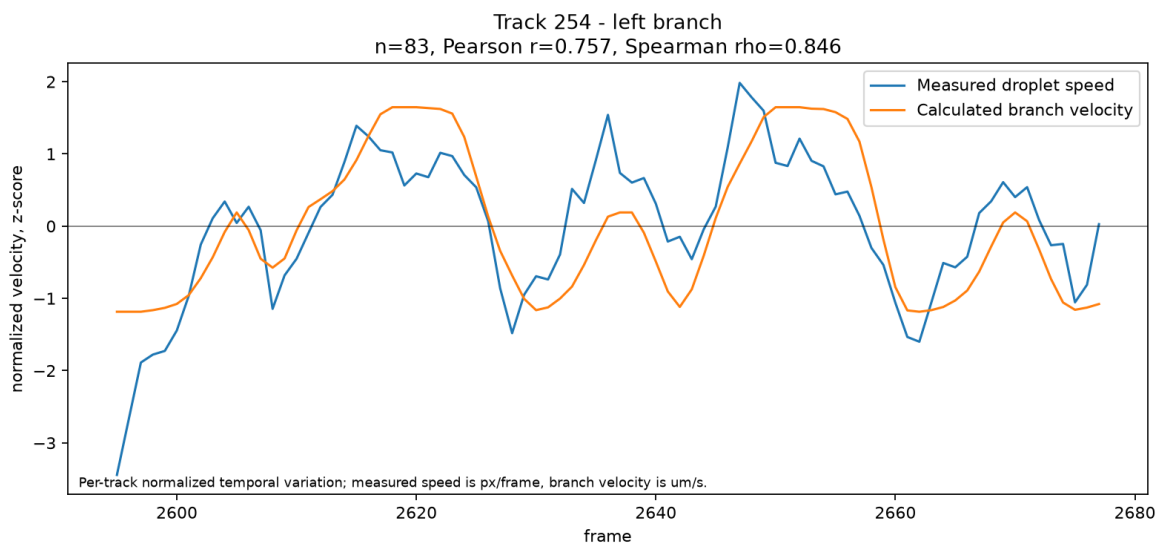


Figure 28: Selected left-branch validation track 254.

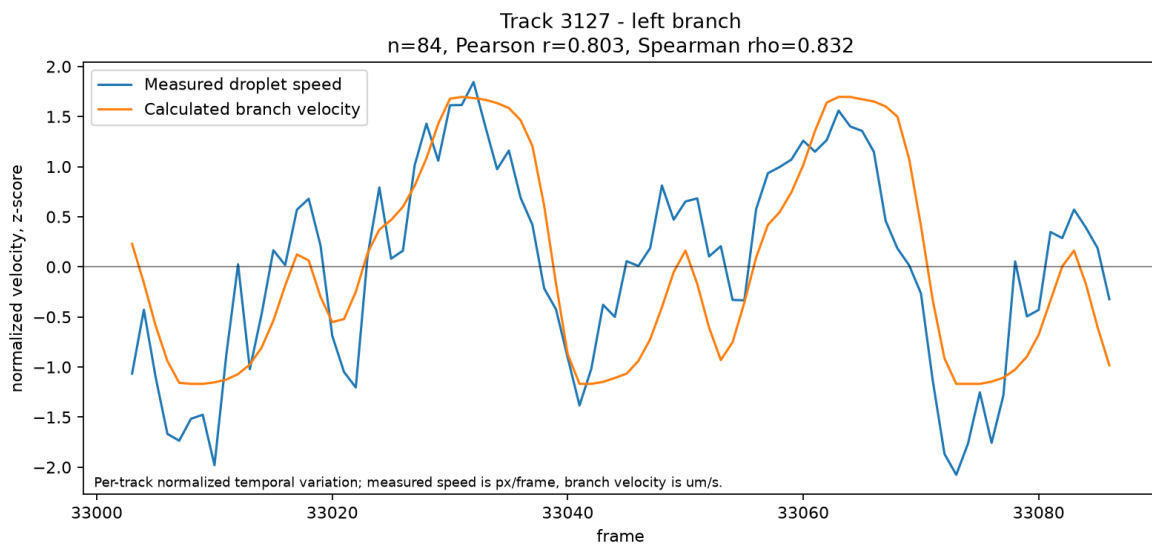


Figure 29: Selected left-branch validation track 3127.

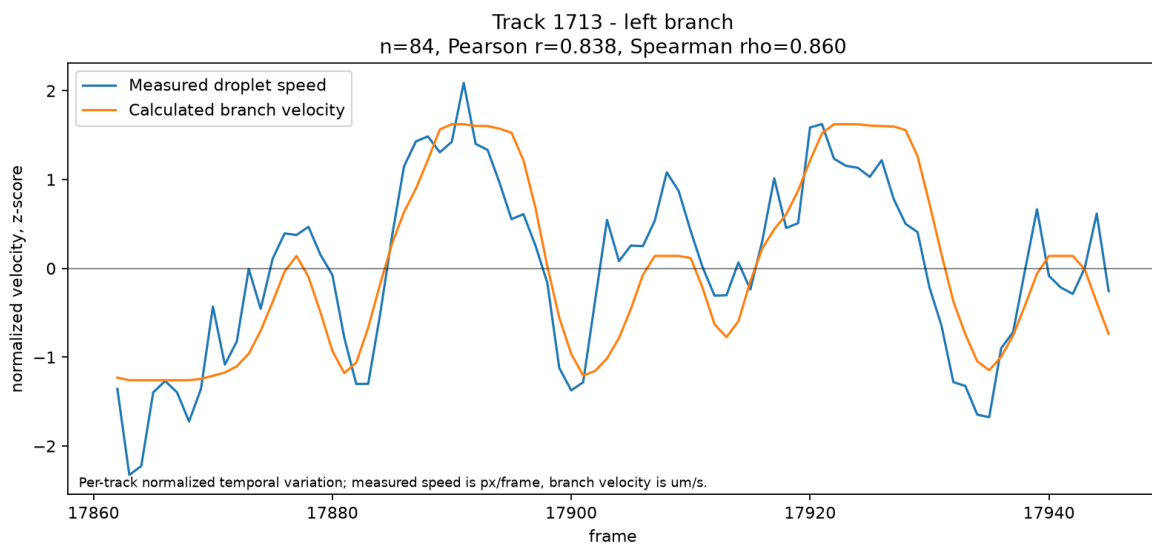


Figure 30: Selected left-branch validation track 1713.

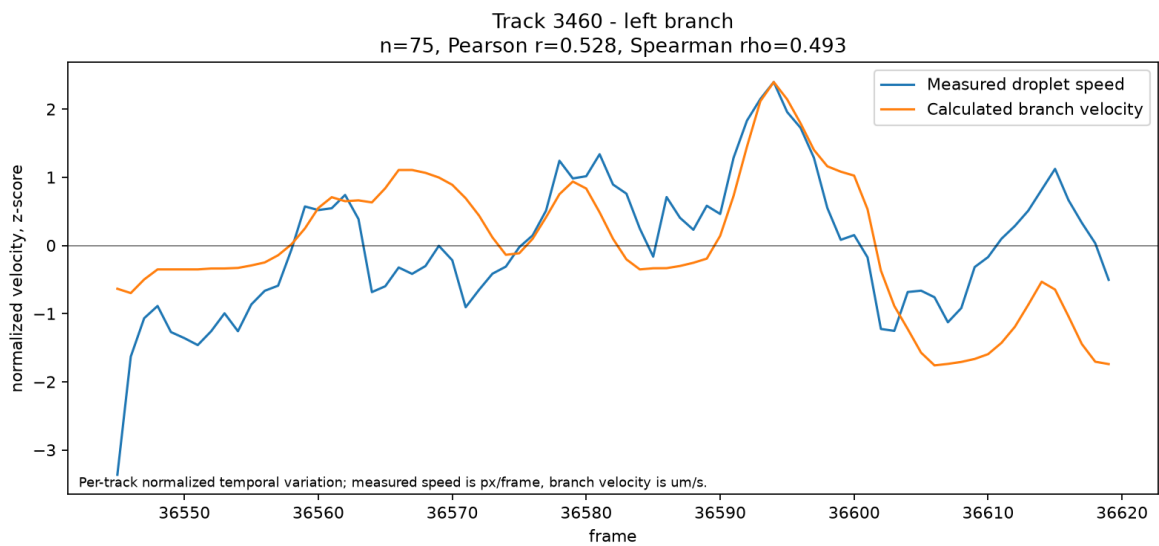


Figure 31: Selected left-branch validation track 3460.

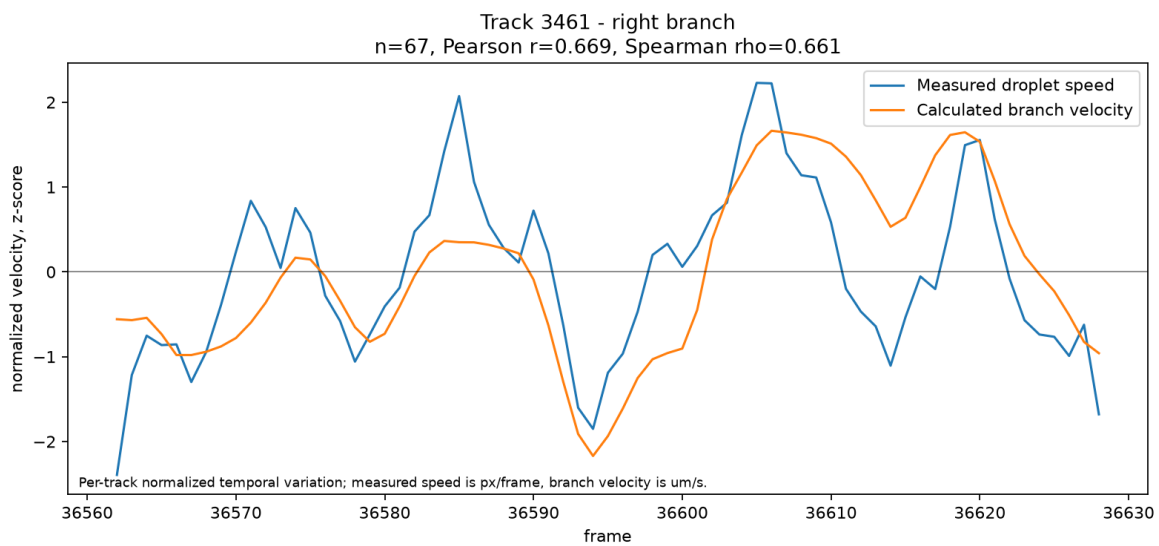


Figure 32: Selected right-branch validation track 3461.

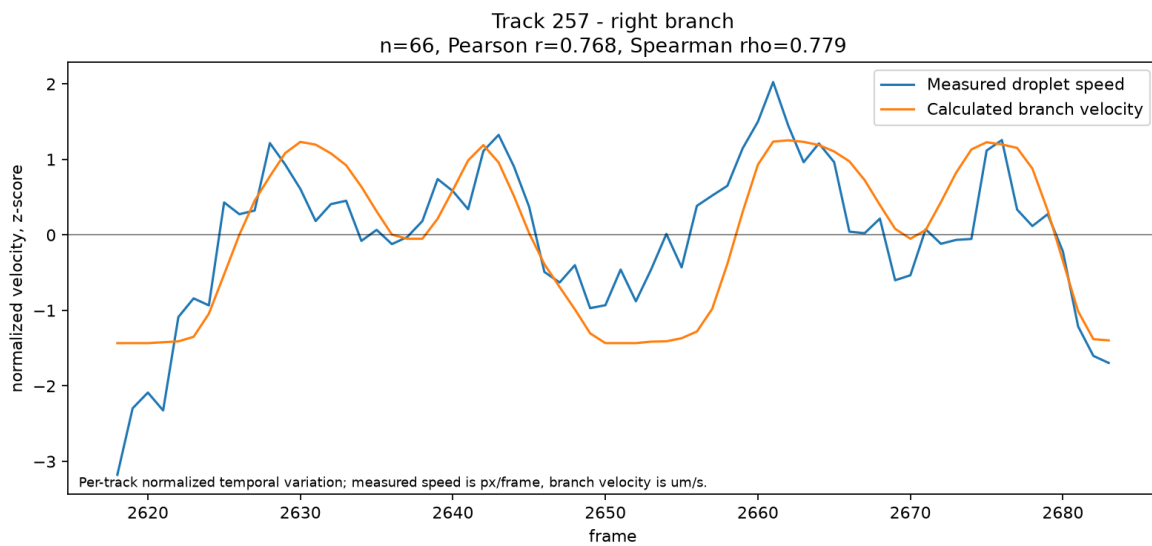


Figure 33: Selected right-branch validation track 257.

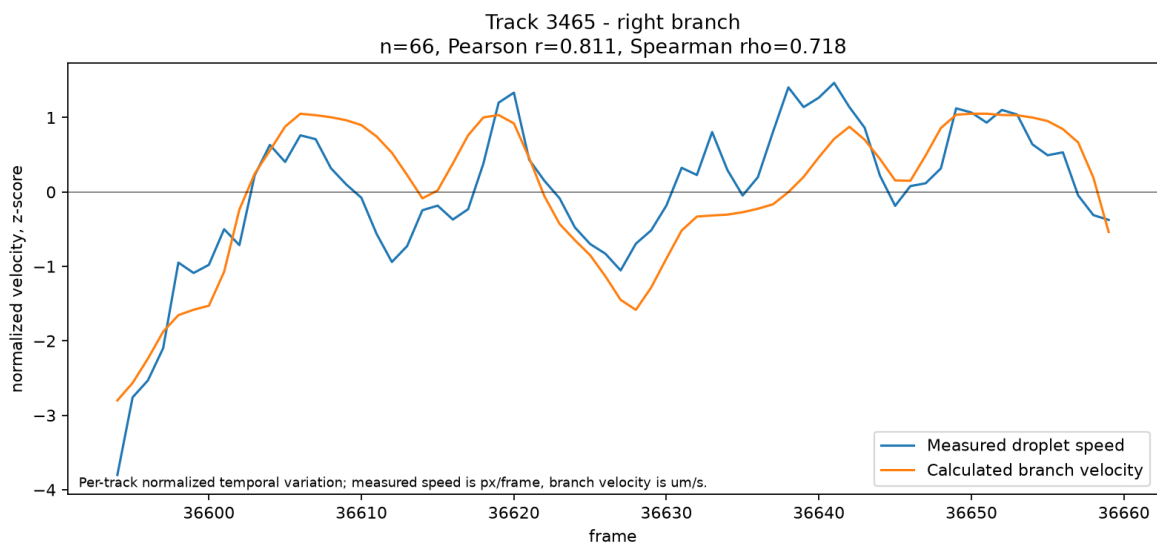


Figure 34: Selected right-branch validation track 3465.

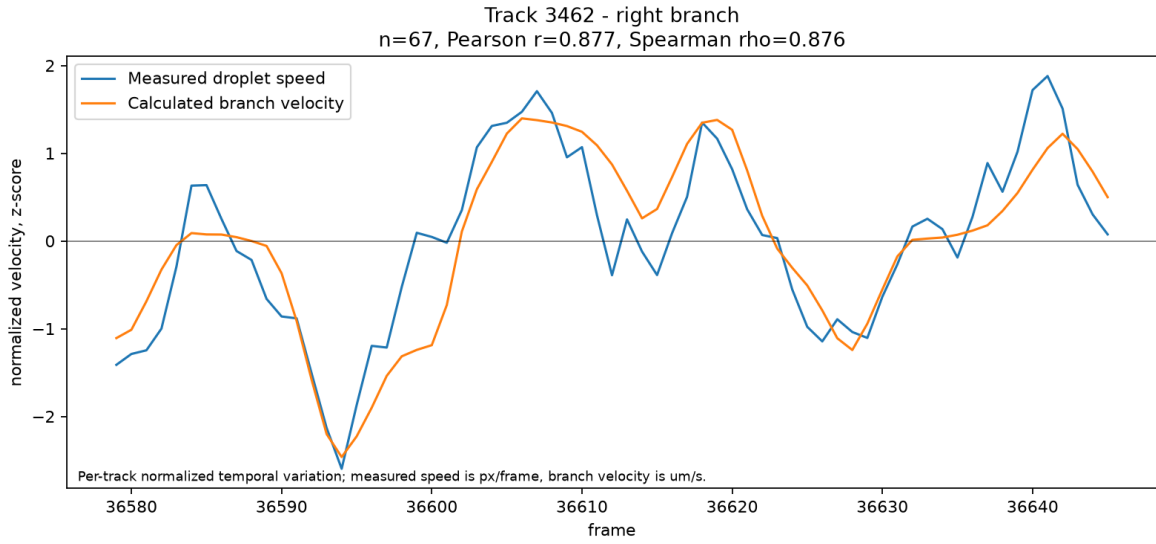


Figure 35: Selected right-branch validation track 3462.

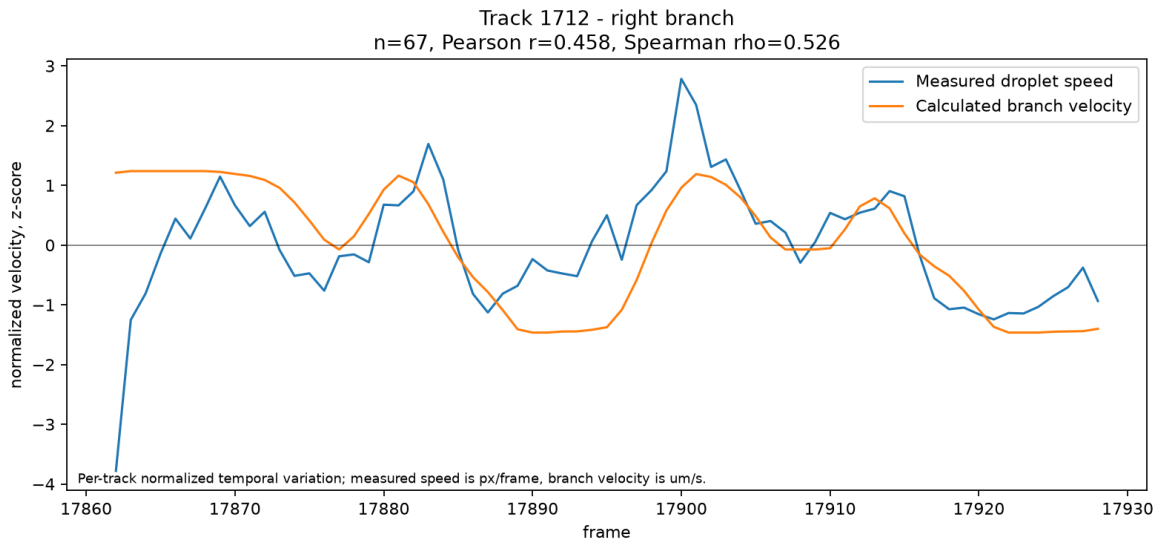


Figure 36: Selected right-branch validation track 1712.

14.9 Execution

```

.\.venv\Scripts\python.exe -m scripts.validation.validate_droplet_branch_velocity \
--experiment configs\experiments\video_2.yml \
--overwrite

```

15 Rollout Model Comparison

15.1 Purpose

The rollout comparison module evaluates whether physics-embedded state improves long-horizon droplet trajectory prediction relative to the two previous Markovian baselines. The comparison is intentionally aligned: all models are evaluated on the same validation windows, with the same droplet slot ordering and the same truth-injection semantics for newly appearing droplets.

15.2 Models Compared

The self-contained evaluation bundle lives under `src/evaluation/`. It contains local copies of the old comparison pipeline, the attention model class, geometry-admissibility helpers, centerline/mask data, and the exact checkpoints used for evaluation.

Label	Input state	Checkpoint
Geometry-naive Markovian	5 features	<code>src/evaluation/models/geometry_naive_markovian/markovian_</code>
Geometry-aware Markovian	5 features	<code>src/evaluation/models/geometry_aware_markovian/geometry_a</code>
Physics-embedded Markovian	15 features	<code>src/evaluation/models/physics_embedded_markovian/physics_</code>

The two previous baselines consume only:

$$(x, y, v_x, v_y, \text{circularity}).$$

The physics-embedded model consumes the full 15-feature state described in the previous section. All three models predict only (v_x, v_y) .

15.3 Rollout Semantics

For each model, predicted velocity is integrated autoregressively:

$$\hat{\mathbf{x}}_{k+1} = \hat{\mathbf{x}}_k + \hat{\mathbf{v}}_k.$$

The comparison preserves the original rollout framework:

- identical validation rollout starts for all models;
- identical selected droplet slots for all models;
- truth injection only for newly appearing droplets;
- no future target velocity or position as model input;
- 50-step rollout horizon;
- 1000 bootstrap replicates for confidence intervals.

For old models, the evaluator slices the current canonical state down to the first five physical/tracking features. For the physics model, it passes the full 15-feature tensor and uses the same non-target feature refresh rule as the physics attention visualizer.

15.4 Metrics

The comparison reports:

- per-step position RMSE;
- per-step velocity RMSE;
- integrated position RMSE over the rollout;
- final-step position RMSE;
- tangential and normal error relative to the centerline tangent basis;
- channel footprint outside-fraction and violation rates;
- trajectory-level geometry-violation statistics;
- junction branch-decision outcomes.

Directional decomposition projects position or velocity error onto the local centerline basis:

$$e_{\parallel} = \mathbf{e} \cdot \mathbf{t}, \quad e_{\perp} = \mathbf{e} \cdot \mathbf{n},$$

where $\mathbf{t}$ is the local tangent and $\mathbf{n}$ is the local normal. Channel admissibility uses the droplet bounding ellipse and reports the fraction of its footprint outside the binary channel mask.

15.5 Final Results

The complete comparison used 1174 validation windows and 50 rollout steps. Headline position metrics are:

Model	Integrated RMSE	95% CI low	95% CI high	S50 RMSE
Geometry-naive Markovian	15.683	13.844	17.266	37.082
Geometry-aware Markovian	15.958	14.168	17.646	38.179
Physics-embedded Markovian	1.540	1.449	1.628	2.401

Selected stepwise position RMSE values are:

Model	S1	S5	S10	S20	S30	S40	S50
Geometry-naive Markovian	0.366	1.283	3.090	9.444	18.160	27.827	37.082
Geometry-aware Markovian	0.365	1.271	2.804	9.176	18.483	28.777	38.179
Physics-embedded Markovian	0.264	0.619	0.956	1.415	1.762	2.049	2.401

Junction branch-decision metrics were:

Model	True events	Correct	Wrong branch rate	Noncommitment rate
Geometry-naive Markovian	4990	4513	0.08198	0.01483
Geometry-aware Markovian	4990	4557	0.07958	0.00782

Physics-embedded Markovian	4990	4949	0.00020	0.00802
----------------------------	------	------	---------	---------

Trajectory-level channel-admissibility metrics were:

Model	Evaluable trajectories	Violation fraction	Mean first violation step	Persistent
Geometry-naive Markovian	17842	0.7226	19.90	
Geometry-aware Markovian	17842	0.6007	22.62	
Physics-embedded Markovian	17842	0.4784	24.42	

15.6 Figures

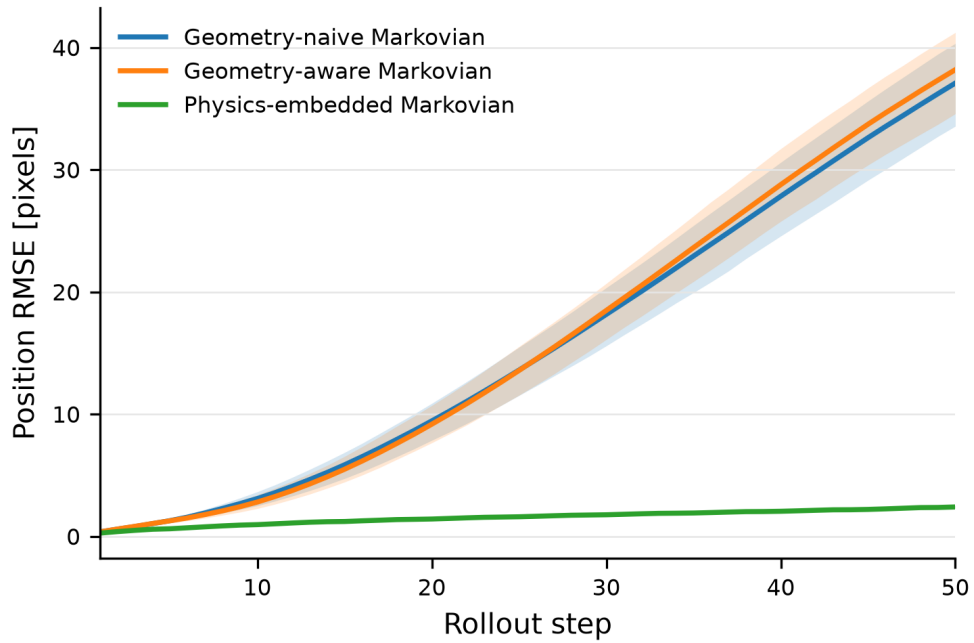


Figure 37: Position RMSE over a 50-step rollout for the two previous Markovian baselines and the physics-embedded Markovian model.

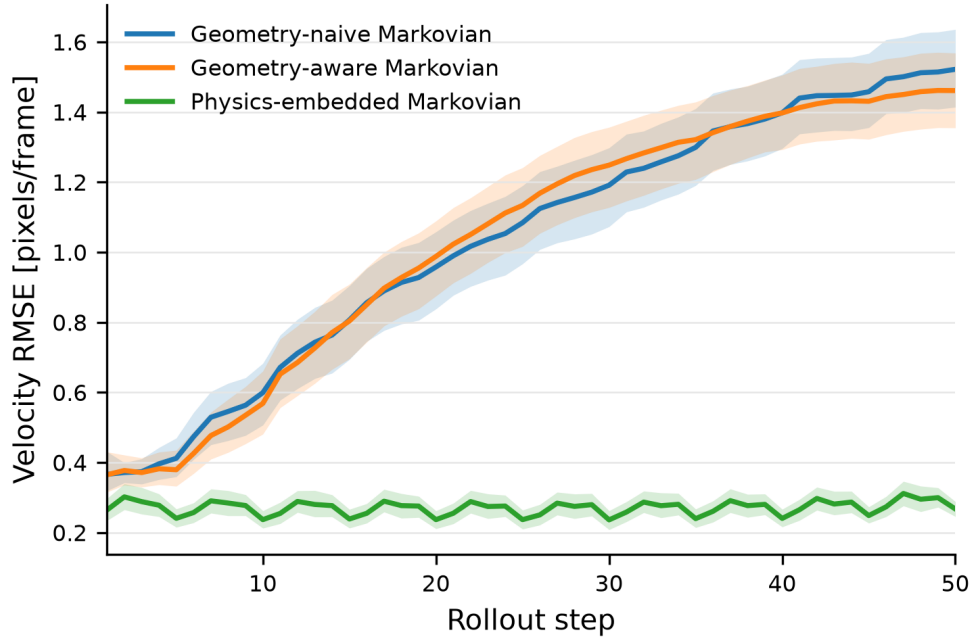


Figure 38: Velocity RMSE over the same aligned validation windows.

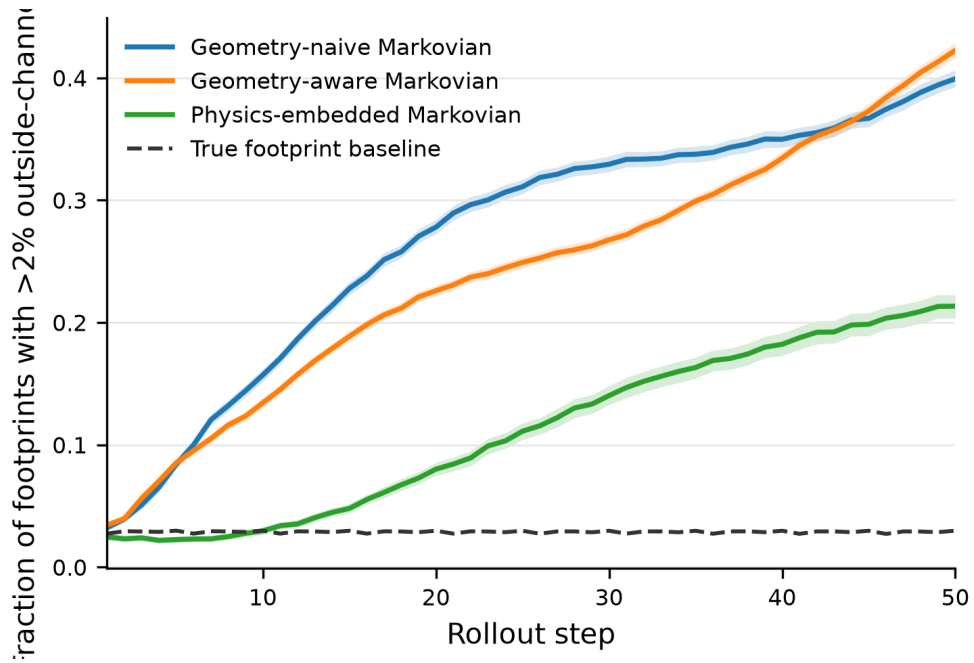


Figure 39: Fraction of predicted droplet footprints with more than 2% of the ellipse outside the channel mask.

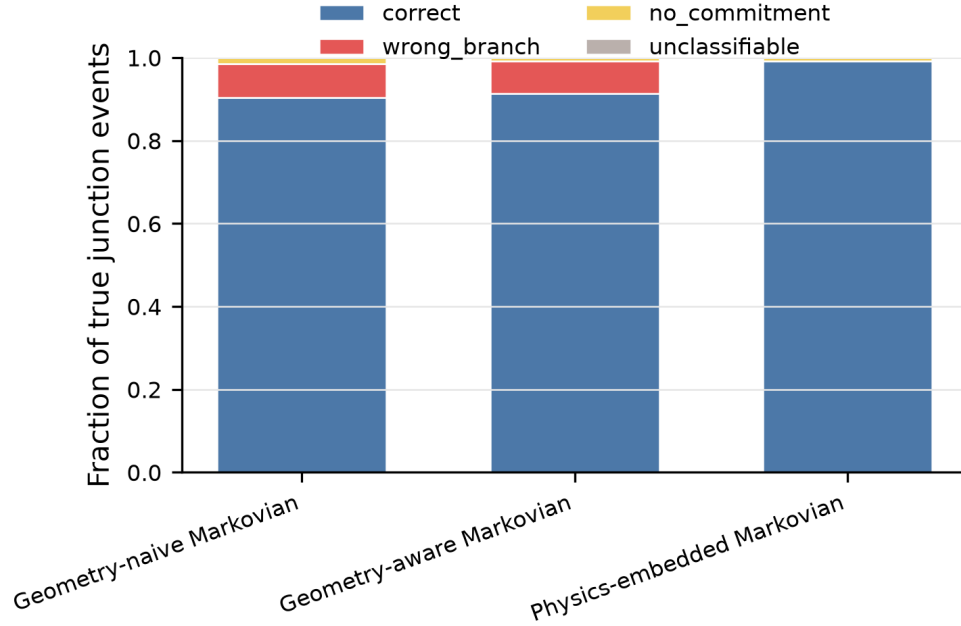


Figure 40: Junction branch-decision outcomes for true junction events.

15.7 Execution

The completed local resume command was:

```
python -m src.evaluation.rollout_comparison \
  --predictions-path src/evaluation/outputs/outputs/rollout_model_comparison/rollout_predictions \
  --output-dir src/evaluation/outputs/rollout_model_comparison \
  --cpu
```

The clean final output directory is:

```
src/evaluation/outputs/rollout_model_comparison
```

16 Testing and Execution

16.1 Test Suite

The test suite covers:

- configuration-to-device resolution;
- centerline ordering, orientation, arc length, tangent, and normal behavior;
- device region-label construction and connectivity;
- droplet ellipse rasterization and occupancy normalization;
- baseline branch-hydraulic equations and flow conservation;
- centered finite-difference velocity validation;

- full-device CFD geometry, mesh generation, solver assembly, and field-level mesh verification;
- canonical dataset v2 construction and physics-enabled window loading;
- full-device CFD interpolation and physics-enriched tracking;
- physics Markovian trainer smoke-test behavior when PyTorch is available.

Run all tests with:

```
.\.venv\Scripts\python.exe -m pytest tests
```

The latest full repository test run before the training/evaluation additions passed:

173 passed, 1 skipped

The skipped test is PyTorch-gated in the local environment; the physics Markovian smoke path was validated with a PyTorch environment separately.

16.2 Primary Commands

1. Build calibrated device geometry:

```
.\.venv\Scripts\python.exe -m scripts.geometry.build_geometry \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```

2. Build device region labels:

```
.\.venv\Scripts\python.exe -m scripts.geometry.build_device_regions \
  --experiment configs\experiments\video_2.yml \
  --frame-index 0 \
  --overwrite
```

3. Compute droplet occupancy:

```
.\.venv\Scripts\python.exe -m scripts.occupancy.compute_droplet_occupancy \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```

4. Compute baseline hydraulics:

```
.\.venv\Scripts\python.exe -m scripts.physics.compute_baseline_hydraulics \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```

5. Run branch-velocity validation:

```
.\.venv\Scripts\python.exe -m scripts.validation.validate_droplet_branch_velocity \
  --experiment configs\experiments\video_2.yml \
  --overwrite
```

6. Build full-device CFD mesh:

```
.\.venv\Scripts\python.exe -m scripts.physics.build_full_device_mesh
```

7. Run full-device scalar mesh convergence:

```
$env:MPLCONFIGDIR='outputs\.matplotlib'  
.\.venv\Scripts\python.exe scripts\physics\run_full_device_convergence.py '  
--target-sizes-um 48 36 24 18 12 '  
--output-dir outputs\physics\full_device_cfd\convergence_alpha0_fine
```

8. Verify the frozen 24 μm full-device field:

```
$env:MPLCONFIGDIR='outputs\.matplotlib'  
.\.venv\Scripts\python.exe scripts\physics\verify_full_device_field_convergence.py '  
--grid-spacing-um 6 '  
--output-dir outputs\physics\full_device_cfd\convergence\field_comparison_24_vs_12
```

9. Train the physics-enabled Markovian rollout model:

```
python -m scripts.training.train_physics_markovian \  
--config configs/experiments/physics_markovian_v1.yml
```

10. Generate physics Markovian attention overlays:

```
python -m scripts.training.visualize_physics_markovian_attention \  
--checkpoint outputs/models/physics_markovian_v1/latest_checkpoint.pt \  
--window-index 125 \  
--target-slot 4 \  
--length 100
```

11. Run the three-model rollout comparison:

```
python -m src.evaluation.rollout_comparison \  
--output-dir src/evaluation/outputs/rollout_model_comparison
```

16.3 Current Scope Boundaries

The project currently does not implement:

- hydraulic parameter fitting;
- droplet slip modeling;
- junction loss modeling.